

Introdução ao método dos elementos de contorno

Notas de aula

Lucas S. Campos

Materiais complementares:

[Apostila](#) · [Entregas](#) · [Playlist YouTube](#)

Sumário

1	Apresentação	4
1.1	Método dos elementos de contorno	4
1.2	Vantagens e desvantagens	4
1.3	Por que JULIA?	5
1.4	Formulação do BEM-1d	6
1.5	Equação de Laplace	7
1.6	Exercícios	9
2	Interpolação	10
2.1	Interpolação polinomial	10
2.2	Exercícios	11
3	Continuando com interpolação	12
3.1	Interpolação por polinômios por partes	13
3.2	Exercício	13
3.3	Estabilidade da interpolação polinomial	13
3.4	Fenômeno de Runge	14
3.5	Integração numérica	15
3.6	Integrais singulares ou quasi-singulares	19
3.7	Desafio	21
4	Equações diferenciais	21
4.1	Sistemas de equações	24
4.2	Runge-Kutta	26
4.3	Métodos de múltiplos passos	29
4.4	Matrizes de diferenças finitas	33
4.5	A equação de difusão	35
4.6	Exercício -BEM x MDF	35
4.7	Equação da onda	35
4.8	Desafio	36
5	Viga de Euler	37
5.1	Teoria de vigas	37
6	Efeitos transientes	39
7	Desafio	40
8	Indo para 2D	41
8.1	Cálculo do perímetro de figuras planas	41
8.2	Área	44
9	Laplace 2D	49
10	Formulação	49
10.1	Método indireto para o cálculo da diagonal da matriz H	50
10.2	Exemplo	50
10.3	Pontos internos	51
10.4	Exercícios	52
10.5	Desafio	54
11	Medidas de erro	54
12	Poisson 2D	55
13	MECID	55
13.1	Exercícios	56
13.2	Gravação	58
14	Elasticidade 2D	58

15	Equações importantes	58
15.1	Exercícios	60
16	Propgeo 3D	63

1 Apresentação

1.1 Método dos elementos de contorno

O método dos elementos de contorno é bem conhecido entre engenheiros e cientistas. Ele tem demonstrado sua superioridade em relação a outros métodos numéricos, especialmente quando usado para modelar uma aplicação apropriada. Apesar da popularidade do método dos elementos de contorno, ele não é tão comum entre os engenheiros quanto o método dos elementos finitos. As razões para isso podem ser resumidas da seguinte forma:

1- A complexidade da formulação matemática. 2- A falta de códigos simples. 3- A falta de cursos de elementos de contorno entre estudantes de graduação. 4- A dificuldade no tratamento de alguns modelos numéricos, como singularidade. 5- A dificuldade em modificar programas de elementos de contorno em relação aos desenvolvidos usando elementos finitos. 6- A falta de versatilidade dos códigos de elementos de contorno. 7- A mudança de estratégia de modelagem de elementos finitos para elementos de contorno.

1.2 Vantagens e desvantagens

O Método dos Elementos de Contorno (BEM), como qualquer outro método numérico, tem suas vantagens e desvantagens. As vantagens do método dos elementos de contorno são as seguintes:

1- Apenas o contorno do problema precisa ser discretizado, o que leva a uma fácil preparação de dados e menores requisitos de computação. 2- O tratamento exato de domínios infinitos e semi-infinitos. 3- As incógnitas em locais internos são calculadas na fase de pós-processamento, o que simplifica qualquer procedimento de otimização. 4- Resultados precisos no caso de concentrações de tensões devido a fissuras ou cargas concentradas.

Por outro lado, as desvantagens do método dos elementos de contorno são as seguintes: 1- As matrizes do sistema são não simétricas e totalmente preenchidas. 2- As soluções fundamentais nem sempre são fáceis de obter. 3- A dificuldade em tratar estruturas delgadas. 4- A discretização do domínio necessária no caso de aplicações não lineares.

Desafios que precisam ser enfrentados para que o método tenha uma maior adesão:

1- Uma visão mais profunda dos aspectos matemáticos e numéricos do método. 2- Um método sistemático para a derivação das soluções fundamentais e particulares. 3- Fórmula de integração estável. 4- Programas de uso geral e pequenos programas disponíveis para engenheiros. 5- Acoplamento entre elementos de contorno e elementos finitos.



1.3 Por que JULIA?

A escolha da linguagem JULIA é justificada por várias razões. JULIA é uma linguagem de programação de alto desempenho, projetada especificamente para computação científica e análise numérica. Suas bibliotecas e funcionalidades são otimizadas para realizar cálculos complexos de forma eficiente, o que é crucial para a implementação de métodos como o Método dos Elementos de Contorno (BEM). Além disso, JULIA possui uma sintaxe simples e intuitiva, o que facilita a escrita e a leitura de códigos, tornando o processo de desenvolvimento mais ágil.

Desempenho: JULIA é projetada para ter um desempenho superior em cálculos numéricos e científicos, superando Python em termos de velocidade. Isso se deve à sua capacidade de compilar código em tempo de execução, o que resulta em execução mais rápida. Python, por outro lado, pode ser mais lento devido à sua natureza interpretada, embora existam bibliotecas como Cython que ajudam a melhorar o desempenho.

Normalmente, uma linguagem é utilizada para o desenvolvimento rápido e prototipagem, enquanto a outra é usada para obter desempenho otimizado. Por exemplo, um cientista pode usar Python para escrever a lógica do código devido à sua simplicidade e riqueza de bibliotecas, mas precisará reescrever partes críticas em C ou Fortran para alcançar a velocidade necessária.

Essa abordagem tem várias desvantagens:

1. **Complexidade de Manutenção:** Manter código em duas linguagens diferentes pode ser complexo e propenso a erros, especialmente quando as mudanças precisam ser sincronizadas entre as duas versões.
2. **Curva de Aprendizado:** Exige que os desenvolvedores sejam proficientes em ambas as linguagens, o que pode não ser sempre o caso.
3. **Integração Difícil:** A integração entre diferentes linguagens pode ser complicada, exigindo ferramentas e técnicas adicionais para gerenciar a comunicação entre elas.

Uma solução para esse problema é a utilização de linguagens como JULIA, que são projetadas para oferecer tanto facilidade de uso quanto alto desempenho. Isso elimina a necessidade de usar duas linguagens diferentes, simplificando o desenvolvimento, a manutenção e a execução de programas complexos.

Para instalar abra o terminal do windows e rode:

```
winget install julia -s msstore
julia
```

1.4 Formulação do BEM-1d

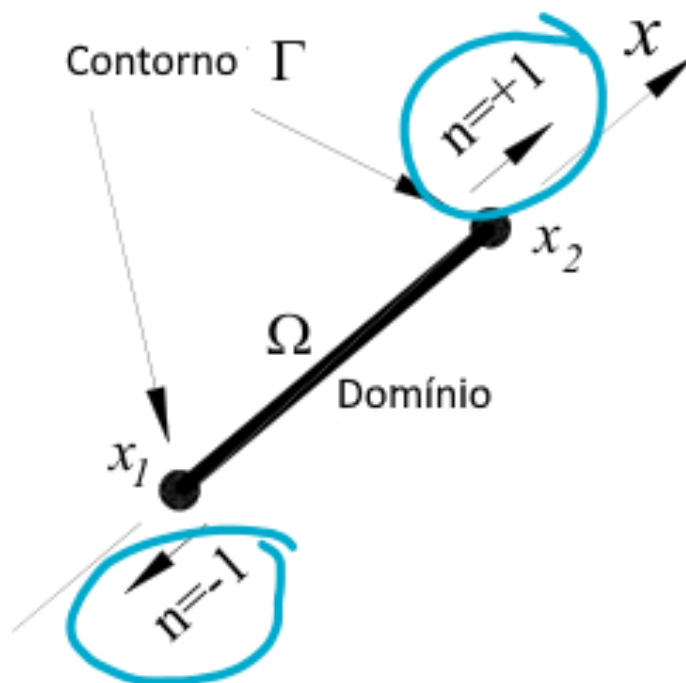
Considere u e v como duas funções da variável independente x em um espaço unidimensional. A seguinte fórmula de integração por partes é bem conhecida:

$$\int_{x=x_1}^{x=x_2} u(x)dv(x) = [u(x)v(x)]_{x=x_1}^{x=x_2} - \int_{x=x_1}^{x=x_2} v(x)du(x)$$

ela pode ser rescrita de maneira simplificada como:

$$\int_{x=x_1}^{x=x_2} u(x)v'(x)dx = [u(x)v(x)]_{x=x_1}^{x=x_2} - \int_{x=x_1}^{x=x_2} v(x)u'(x)dx$$

onde $'$ representa a derivada. O primeiro termo do lado direito pode ser rescrito levando em conta o **contorno**. No caso unidimensional isso consiste exatamente nos pontos x_1 e x_2 .



$$\begin{aligned} [u(x)v(x)]_{x=x_1}^{x=x_2} &= [u(x)v(x)n(x)]_{x=x_2} + [u(x)v(x)n(x)]_{x=x_1} \\ &\Rightarrow \sum_{x=x_1, x_2} u(x)v(x)n(x) \Rightarrow \int_{\Gamma} u(x)v(x)n(x)d\Gamma \end{aligned}$$

Os demais termos tratam de uma integral no domínio e pode ser escritas como:

$$\int_{\Omega} u(x)v'(x)d\Omega = \int_{\Gamma} u(x)v(x)n(x)d\Gamma - \int_{\Omega} v(x)u'(x)d\Omega$$

É importante notar: 1- A ideia principal da integração por partes, é trocar o operador diferencial da função v para a função u . 2- Ao fazer essa troca, alguns termos de contorno aparecem. 3- A integração por partes foi feita apenas uma vez. No entanto, no BEM a integração por partes pode ser realizada uma, duas ou até quatro vezes dependendo da ordem da derivada. 4- Esse processo pode ser facilmente estendido para 2 ou 3 dimensões.

1.5 Equação de Laplace

A formulação diferencial da equação de Laplace é dada por:

$$\frac{d^2T(x)}{dx^2} = 0$$

Usando integração por partes duas vezes na expressão dos resíduos ponderados:

$$\int_{x_0}^{x_f} T^* \frac{d^2T}{dx^2} dx = \left(T^* \frac{dT}{dx} \right)_{x_0}^{x_f} - \int_{x_0}^{x_f} \frac{dT^*}{dx} \frac{dT}{dx} dx = \left(T^* \frac{dT}{dx} - T \frac{dT^*}{dx} \right)_{x_0}^{x_f} - \int_{x_0}^{x_f} \frac{d^2T^*}{dx^2} T dx.$$

Reescrevendo em termos do fluxo $Q = \frac{dT}{dx}$:

$$\int_{x_0}^{x_f} T^* \frac{d^2T}{dx^2} dx = -(TQ^* - T^*Q)_{x_0}^{x_f} - \int_{x_0}^{x_f} \frac{d^2T^*}{dx^2} T dx.$$

Considerando que T^* é a solução fundamental do problema, ou seja:

$$-\frac{d^2T^*(x, x_d)}{dx^2} = \delta(x - x_d).$$

onde δ é a função delta de Dirac. Pode-se observar que $T^* = -\frac{1}{2} |x - x_d|$ e $Q^* = \frac{dT^*}{dx} = -\frac{1}{2} \text{sign}(x - x_d)$. Substituindo essas funções na equação integral obtém-se:

$$-T(x_d) = T(x_f)Q^*(x_f, x_d) - T^*(x_f, x_d)Q(x_f) - T(x_0)Q^*(x_0, x_d) + T^*(x_0, x_d)Q(x_0).$$

Quando x_d é x_0 tem-se:

$$\begin{aligned} T^*(x_0, x_0) &= 0 \\ T^*(x_f, x_0) &= -\frac{x_f - x_0}{2} \\ Q^*(x_0, x_0) &= \frac{1}{2} \\ Q^*(x_f, x_0) &= -\frac{1}{2} \end{aligned}$$

Quando x_d é x_f tem-se:

$$T^*(x_0, x_f) = -\frac{x_f - x_0}{2}$$

$$T^*(x_f, x_f) = 0$$

$$Q^*(x_0, x_f) = \frac{1}{2}$$

$$Q^*(x_f, x_f) = -\frac{1}{2}$$

Logo, para o ponto $x_d = x_0$ a equação se torna:

$$-T(x_0) = -T(x_f)\frac{1}{2} + \frac{x_f - x_0}{2}Q(x_f) - T(x_0)\frac{1}{2} + 0Q(x_0)$$

e para o ponto $x_d = x_f$

$$-T(x_f) = -T(x_f)\frac{1}{2} - 0Q(x_f) - T(x_0)\frac{1}{2} - \frac{x_f - x_0}{2}Q(0)$$

Escrevendo as duas equações em forma matricial, tem-se:

$$\begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix} \begin{bmatrix} T(0) \\ T(1) \end{bmatrix} = (x_f - x_0) \begin{bmatrix} 0 & -0.5 \\ 0.5 & 0 \end{bmatrix} \begin{bmatrix} Q(0) \\ Q(1) \end{bmatrix}.$$

1.5.1 Exemplos

Caso 1: Condição de Dirichlet $T(0) = 100, T(1) = 0$ $T(x) = 100 - 100x$

Caso 2: Condição mista $T(0) = 100, Q(0) = 0$ $T(x) = 100$

```
x0=0
xf=1
l=xf-x0
A1=[0.5 -.5 0 l/2
    -.5 .5 -l/2 0
    1 0 0 0
    0 1 0 0]
b=[0,0,100,0]
x1=A1\b

A2=[0.5 -.5 0 l/2
    -.5 .5 -l/2 0
    1 0 0 0
    0 0 1 0]
b=[0,0,100,0]
x2=A2\b
```

Usando essa equação podemos calcular o valor da temperatura nos pontos internos:

```
x0=0
xf=1
xs=range(x0,xf,length=100)
T=0.5*(x1[1]+x1[2]).+(xs.-x0)/2*x1[3].+(xs.-xf)/2*x1[4]
using Plots
plot(xs,T,legend=false,xlabel="x",ylabel="T",marker=:c)
```


1.6 Exercícios

1- Uma outra possível condição de contorno é a convecção:

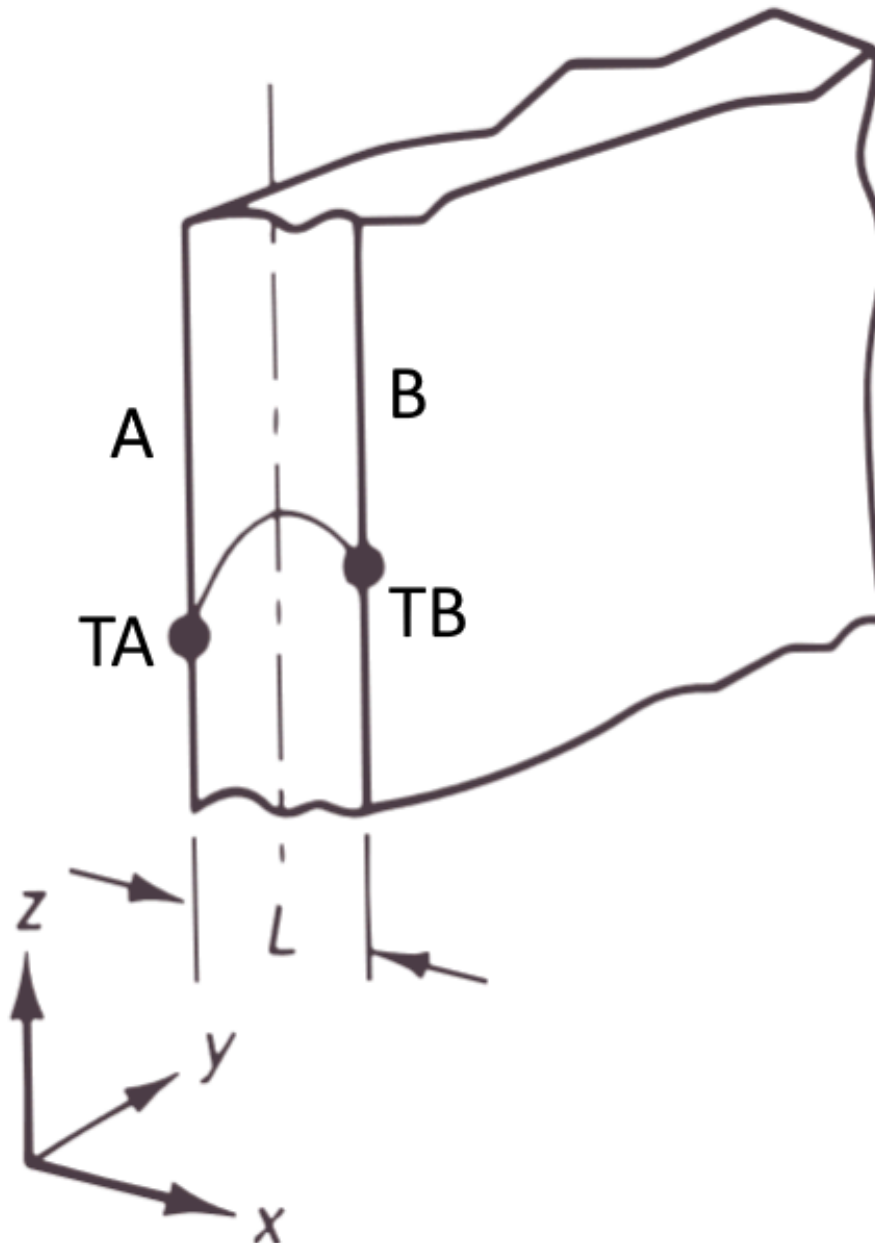
$$q = h(T - T_{\infty})$$

considere o problema onde o lado direito está exposto a $T_{\infty} = 20^{\circ}\text{C}$, no lado esquerdo $T(0) = 100^{\circ}\text{C}$, e $h = 3\text{ W/}^{\circ}\text{C}$. Resolva esse problema analiticamente e numericamente usando BEM.

2- Quando existe uma fonte de calor distribuída b mais um termo aparece na equação integral:

$$-T(x_d) = T(x_f)Q^*(x_f, x_d) - T^*(x_f, x_d)Q(x_f) - T(x_0)Q^*(x_0, x_d) + T^*(x_0, x_d)Q(x_0) + \int_{x_0}^{x_f} T^*(x, x_d)b dx.$$

Considere uma carga constante (isso torna a integral restante muito fácil de ser integrada analiticamente) e resolva um problema de uma grande placa de espessura $L = 2\text{ cm}$ com condutividade térmica constante $k = 1\text{ W/m.K}$ e geração uniforme de calor $b = 1000\text{ kW/m}^3$. As faces A e B estão a temperaturas de 100°C e 200°C , respectivamente. Compare com a solução analítica:



$$T = \left[\frac{T_B - T_A}{L} + \frac{b}{2k}(L - x) \right] x + T_A$$

1.6.1 Extra

Quando dois corpos trocam calor por radiação, o fluxo é proporcional à diferença da quarta potência de suas temperaturas absolutas: $q_n = \kappa f_s f_\epsilon (u^4 - u_R^4)$ onde u, u_R são as temperaturas absolutas dos corpos radiantes, $\kappa = 5.699 \times 10^{-8} \text{ W/(m}^2 \text{ K}^4)$ é a constante de Stefan-Boltzmann, $0 \leq f_s \leq 1$ é o fator de forma da radiação e $0 < f_\epsilon \leq 1$ é a emissividade superficial, definida como o poder emissivo relativo de um corpo em comparação ao de um corpo negro ideal. A emissividade superficial também é igual ao coeficiente de absorção, definido como a fração da energia térmica incidente em um corpo que é absorvida. A radiação pode ser vista como uma condição de contorno convectiva onde o coeficiente de transferência de calor convectivo depende da temperatura dos corpos radiantes. Escrevendo:

$$q_n = \kappa f_s f_\epsilon (u^4 - u_R^4) = \underbrace{\kappa f_s f_\epsilon (u^2 + u_R^2)}_{h_r(u)} (u + u_R)(u - u_R)$$

Os problemas de radiação devem ser resolvidos por iteração: primeiro, o problema linear é resolvido, então o coeficiente de transferência de calor convectivo é atualizado e a solução é repetida. O critério de parada é baseado no tamanho da variação de temperatura. Normalmente, são necessárias poucas iterações.

Implemente essa condição de contorno no BEM.

2 Interpolação

2.1 Interpolação polinomial

Suponha que queremos conhecer a população às vezes entre os anos do censo ou estimar populações futuras. Uma técnica é encontrar um polinômio que passa por todos os pontos de dados.

Dado n pontos $(t_1, y_1), \dots, (t_n, y_n)$, onde os t_i são todos distintos, o problema **de interpolação polinomial** é encontrar um polinomial P de grau menor que n tal que $p(t_i) = y_i$ para todos i .

O problema de interpolação polinomial tem uma solução única. Uma vez encontrado o polinômio interpolador, ele pode ser avaliado em qualquer lugar para estimar ou prever valores.

2.1.1 Interpolação como um sistema linear

Dados os dados (t_i, y_i) por $i = 1, \dots, n$, buscamos um polinômio

$$p(t) = c_1 + c_2 t + c_3 t^2 + \dots + c_n t^{n-1},$$

de tal modo que $y_i = p(t_i)$ para todos i . Essas condições são usadas para determinar os coeficientes $c_1, \dots, c_n$:

$$\begin{aligned} c_1 + c_2 t_1 + \dots + c_{n-1} t_1^{n-2} + c_n t_1^{n-1} &= y_1, \\ c_1 + c_2 t_2 + \dots + c_{n-1} t_2^{n-2} + c_n t_2^{n-1} &= y_2, \\ c_1 + c_2 t_3 + \dots + c_{n-1} t_3^{n-2} + c_n t_3^{n-1} &= y_3, \\ &\vdots \\ c_1 + c_2 t_n + \dots + c_{n-1} t_n^{n-2} + c_n t_n^{n-1} &= y_n. \end{aligned}$$

Essas equações formam um sistema linear para os coeficientes c_i :

$$\begin{bmatrix} 1 & t_1 & \dots & t_1^{n-2} & t_1^{n-1} \\ 1 & t_2 & \dots & t_2^{n-2} & t_2^{n-1} \\ 1 & t_3 & \dots & t_3^{n-2} & t_3^{n-1} \\ \vdots & \vdots & & \vdots & \vdots \\ 1 & t_n & \dots & t_n^{n-2} & t_n^{n-1} \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix},$$

ou simplesmente, $Vc = y$. A Matrix V é de um tipo especial chamado de Vandermonde.

A interpolação polinomial pode, portanto, ser formulada como um sistema linear de equações com uma matriz de Vandermonde. Criamos dois vetores para dados sobre a população da China. O primeiro tem os anos dos dados do censo e o outro tem a população, em milhões de pessoas.

```
year = [1982, 2000, 2010, 2015];
pop = [1008.18, 1262.64, 1337.82, 1374.62];
t = year .- 1980.0
y = pop;
V = [ t[i]^j for i=1:4, j=0:3 ]
#4x4 Matrix{Float64}:
#1.0  2.0  4.0  8.0
#1.0  20.0  400.0  8000.0
#1.0  30.0  900.0  27000.0
#1.0  35.0  1225.0  42875.0
c = V \ y
#4-element Vector{Float64}:
# 962.2387878787877
# 24.12775468975476
# -0.592262049062053
# 0.006843867243867301

using Polynomials
p = Polynomial(c)
p(2005-1980)
```

O valor oficial da população para 2005 foi 1303,72, então nosso resultado é bastante bom.

```
using Plots
scatter(t,y, label="real", legend=:topleft,
xlabel="anos desde 1980", ylabel="população (milhões)",
title="População da China")

# interpole em 500 pontos no intervalo [0,35].
tt = range(0,35,length=500)
# avalie o polinômio nesses pontos
yy = p.(tt)
plot!(tt,yy,label="interpolante")
```

2.2 Exercícios

- Suponha que você queira interpolar os pontos $(-1,0)$, $(0,1)$, $(2,0)$, $(3,1)$ e $(4,2)$ por um polinômio de grau o mais baixo possível. **(a)** 🍌 Qual é o grau máximo necessário desse polinômio? **(b)** 🍌 Escreva um sistema linear de equações para os coeficientes do polinômio interpolador. **(c)** 🖨 Use Julia para resolver numericamente o sistema em (b).

2. (a) 🚀 Suponha que você quer encontrar um polinômio cúbico p tal que $p(-1) = -2$, $p'(-1) = 1$, $p(1) = 0$, e $p'(1) = -1$. (Isso é conhecido como um *Interpolador de Hermite*.) Escreva um sistema linear de equações para os coeficientes de p . (b) 📖 Use Julia para resolver o sistema linear na parte (a), e faça um gráfico de p sobre $-1 \leq x \leq 1$.

3 Continuando com interpolação

Dado $n + 1$ pontos distintos $(t_0, y_0), (t_1, y_1), \dots, (t_n, y_n)$, com $t_0 < t_1 < \dots < t_n$ chamados de nós, o problema de interpolação é encontrar uma função $p(x)$, chamada de interpolante, tal que $p(t_k) = y_k$ para $k = 0, \dots, n$.

Aqui t_k são os nós e x denota a variável independente contínua.

Os nós de interpolação são numerados de 0 a n . Isso é conveniente para nossas declarações matemáticas, mas menos em um idioma como Julia, no qual os índices vetoriais começam com 1. Os índices em um código de computador têm o mesmo significado que os nomeados idênticos nas fórmulas matemáticas e portanto, deve ser incrementado por um sempre que usado em um contexto de indexação.

Os polinômios são o primeiro candidato óbvio a servir como funções interpolando. Eles são fáceis de trabalhar e, em vimos que um sistema linear de equações pode ser usado para determinar os coeficientes de um polinômio que passa por todos os membros de um conjunto de pontos. No entanto, não é difícil encontrar exemplos para os quais a interpolação polinomial leva a resultados inutilizáveis.

Aqui estão alguns pontos que poderíamos considerar ser observações de uma função desconhecida em $[-1, 1]$.

```
n = 5
t = range(-1, 1, length=n+1)
y = @. t^2 + t + 0.05*sin(20*t)

scatter(t, y, label="dados", leg=:top)
```

O interpolante polinomial, calculado usando o `fit`, parece muito bom.

```
p = Polynomials.fit(t, y, n)
plot!(p, -1, 1, label="interpolant")
```

Mas agora considere um conjunto diferente de pontos gerados quase exatamente da mesma maneira.

```
n = 18
t = range(-1, 1, length=n+1)
y = @. t^2 + t + 0.05*sin(20*t)

scatter(t, y, label="data", leg=:top)
```

Os pontos em si não tem nada de especial. Mas dê uma olhada no que acontece com a interpolante polinomial.

```
p = Polynomials.fit(t, y, n)
x = range(-1, 1, length=1000)
plot!(x, p.(x), label="interpolant")
```

Certamente deve haver funções que são mais representativas desses pontos!

3.1 Interpolação por polinômios por partes

Para manter pequenos graus de polinomial enquanto interpolam grandes conjuntos de dados, escolheremos interpolantes dos polinômios por partes. Especificamente, o interpolante p deve ser um polinômio em cada subintervalo $[t_{k-1}, t_k]$ para $k = 1, \dots, n$.

Geralmente, designamos antecipadamente um grau máximo para cada parte polinomial de $p(x)$.

```
scatter(t,y,label="data",leg=:top)
using Dierckx
p1 = Spline1D(t,y;k=1)#grau 1
p2 = Spline1D(t,y;k=2)#grau 2
p3 = Spline1D(t,y;k=3)#grau 3
plot!(x->p1(x),-1,1,label="piecewise linear")
plot!(x->p2(x),-1,1,label="piecewise quadrático")
plot!(x->p3(x),-1,1,label="piecewise cúbico")
```

3.2 Exercício

1. Os dois vetores a seguir definem uma forma geométrica.

```
x = [ 0,0.51,0.96,1.06,1.29,1.55,1.73,2.13,2.61,
      2.19,1.76,1.56,1.25,1.04,0.58,0 ]
y = [ 0,0.16,0.16,0.43,0.62,0.48,0.19,0.18,0,
      -0.12,-0.12,-0.29,-0.30,-0.15,-0.16,0 ]
```

Podemos considerar x e y como funções de um parâmetro s , com os pontos sendo valores dados em $s = 0, 1, \dots, 15$.

(a) Interpole os pontos usando `spline1d` e plote a imagem.

(b) Uma desvantagem do resultado na parte (a) é o canto perceptível no lado esquerdo, que corresponde a $s = 0$ de cima e $s = 15$ de baixo. Teste adicionar a palavra-chave `periodic = true` à chamada de `spline1d` e plote o resultado.

3.3 Estabilidade da interpolação polinomial

Escolhemos uma função em relação ao intervalo $[0, 1]$.

```
f = x -> sin(exp(2*x));
plot(f,0,1,label="função",legend=:bottomleft)
t = (0:6)/6
y = f.(t)
scatter!(t,y,label="nós")
p = Polynomials.fit(t,y)
plot!(p,0,1,label="interpolante",title="Interpolante equidistante, n=6")
```

Isso parece bom. Queremos rastrear o comportamento do erro à medida que N aumenta. Estimaremos o erro no interpolante contínuo, amostrando-o em um grande número de pontos e tomando a norma máxima.

```

using LinearAlgebra
n = 5:5:60; err = zeros(size(n))
x = range(0,1,length=2001)      # pontos para medir o erro
for (i,n) in enumerate(n)
    t = (0:n)/n
    y = f.(t)
    p = Polynomials.fit(t,y)
    err[i] = norm( (@. f(x)-p(x)), Inf )
end
plot(n,err,m=:o,title="Erro de interpolação para nós equidistantes",
axis=("n"),yaxis=(log10,"max error"),)

```

O erro diminui inicialmente como seria de esperar, mas começa a crescer. Ambas as fases ocorrem a taxas exponenciais em n , ou seja, $O(k^n)$, aparecendo linear em um gráfico semi-log.

3.4 Fenômeno de Runge

A decepcionante perda de convergência é um sinal de mau condicionamento devido ao uso de nós igualmente espaçados.

```

using Plots, LaTeXStrings, LinearAlgebra

f = x -> 1/(x^2 + 16)
plot(f, -1, 1, title="Função teste", legend=:none)

```

Essa função possui infinitamente muitas derivadas contínuas em toda a linha real e parece fácil de aproximar em $[-1, 1]$. Começamos fazendo interpolação polinomial equispacada para alguns pequenos valores de n .

```

plot(xaxis=(L"x"), yaxis=(log10, L"|f(x)-p(x)|", [1e-20, 1]))

x = range(-1, 1, length=2501)
n = 4:4:12

for (k,n) in enumerate(n)
    t = range(-1, 1, length=n+1)
    y = f.(t)
    p = Polynomials.fit(t,y)
    err = @. abs(f(x)-p(x))
    plot!(x, err, m=(1, :o, stroke(0)), label="grau $n")
end
title!("Erro para graus baixos")

```

A convergência até agora parece bastante boa, embora não seja uniformemente. No entanto, observe o que acontece à medida que continuamos aumentando o grau.

```

n = @. 12 + 15*(1:3)
plot(xaxis=(L"x"), yaxis=(log10, L"|f(x)-p(x)|", [1e-20, 1]))

for (k,n) in enumerate(n)
    t = range(-1, 1, length=n+1)
    y = f.(t)

```

```

p = Polynomials.fit(t,y)
err = @. abs(f(x)-p(x))
plot!(x,err,m=(1,:0,stroke(0)),label="grau $n")
end
title!("Erro para graus altos")

```

A convergência no meio não pode ficar melhor do que a precisão da máquina em relação aos valores da função. Portanto, a manutenção da lacuna crescente entre o centro e as extremidades empurra as curvas de erro exponencialmente rapidamente nas extremidades, destruindo a convergência.

A observação da instabilidade é conhecida como o fenômeno runge. O fenômeno de Runge é uma instabilidade manifestada quando os nós do interpolante são igualmente espaçados e o grau do polinomial aumenta. Significativamente, a convergência observada é estável dentro de uma parte do intervalo. Ao redistribuir os nós de interpolação, sacrificaremos um pouco da convergência na parte do meio, a fim de melhorá-lo perto das extremidades e resgatar o processo globalmente.

Uma família de nós que fornece convergência estável para a interpolação polinomial é os pontos Chebyshev do segundo tipo definido por:

$$t_k = -\cos\left(\frac{k\pi}{n}\right), \quad k = 0, \dots, n.$$

```

plot(label="",xaxis=(L"x"),yaxis=( :log10,L"|f(x)-p(x)|", [1e-20,1]))
x = range(-1,1,length=2001)
for (k,n) in enumerate([4,10,16,40])
    t = [ -cos(pi*k/n) for k in 0:n ]
    y = f.(t)
    p = Polynomials.fit(t,y)
    err = @.abs(f(x)-p(x))
    plot!(x,err,m=(1,:0,stroke(0)),label="degree $n")
end
title!("Erro com os pontos de Chebyshev")

```

A partir do grau 16, o erro está dentro da precisão de máquina e ele permanece lá à medida que n aumenta.

3.4.1 Exercício

1. Para cada caso, calcule o interpolante polinomial usando n nós de Chebyshev do segundo tipo em $[-1, 1]$ por $n = 4, 8, 12, \dots, 60$. Em cada valor de n , calcule o erro (ou seja, $\max |p(x) - f(x)|$) avaliado em 4000 valores de x . Usando uma escala log-linear, plote o erro em função de n e, em seguida, determine uma boa aproximação à constante k de $O(n^{-k})$.

(a) $f(x) = 1/(25x^2 + 1)$ (b) $f(x) = \tanh(5x + 2)$ (c) $f(x) = \cosh(\sin x)$ (d) $f(x) = \sin(\cosh x)$

3.5 Integração numérica

A primitiva de e^x é simples, isso torna a avaliação de $\int_{-1}^1 e^x dx$ pelo teorema fundamental trivial.

```

using FastGaussQuadrature, LinearAlgebra
exato = exp(1)-exp(-1)

x, w = gausslegendre(3)
#([-0.7745966692414834, 0.0, 0.7745966692414834], [0.5555555555555556,

```

```
0.8888888888888888, 0.5555555555555556])
```

```
f(x) = exp(x)
```

```
In = dot(w, f.(x))  
exato-In
```

A abordagem numérica é muito robusta. Por exemplo, $e^{\sin x}$ não tem primitiva conhecida. Mas numericamente, sua integral não é mais difícil de ser calculada.

```
f(x) = exp(sin(x))  
In = dot(w, f.(x))
```

Quando você olha para os gráficos dessas funções, o que é notável é que uma dessas áreas é muito simples, enquanto o outro é analiticamente muito difícil. Do ponto de vista numérico, eles são praticamente o mesmo problema.

```
using Plots, LaTeXStrings  
plot([exp, x->exp(sin(x))], -1, 1, fill=0, layout=(2, 1),  
      xlabel=L"x", ylabel=[L"e^x" L"e^{\sin(x)}"], ylim=[0, 2.7])
```

A integração numérica, que também passa pelo nome mais antigo quadratura, é executado pela combinação de valores do integrando amostrados nos nós. Nesta seção, assumiremos nós igualmente espaçados:

$$t_i = a + ih, \quad h = \frac{b-a}{n}, \quad i = 0, \dots, n.$$

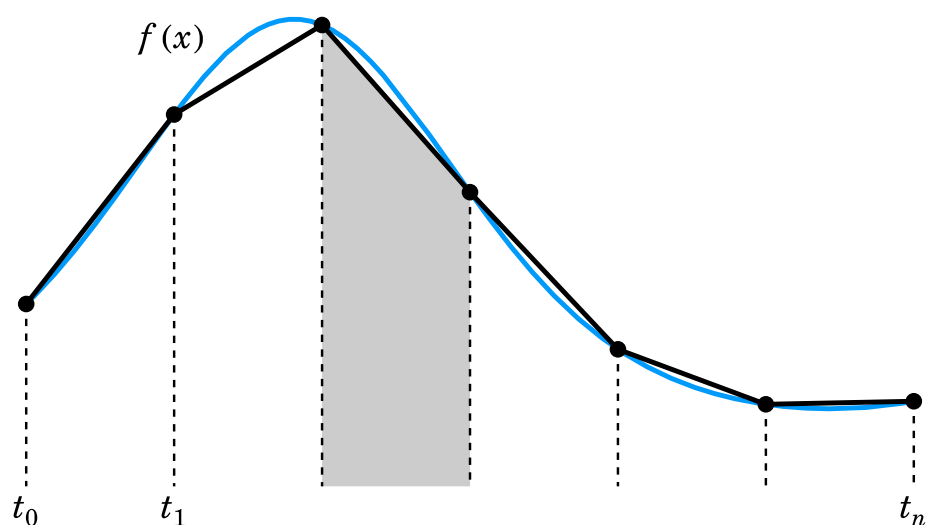
A Integração numérica consiste em uma lista de pesos $w_0, \dots, w_N$ escolhidos de modo que:

$$\int_a^b f(x) dx \approx h \sum_{i=0}^n w_i f(t_i) = h[w_0 f(t_0) + w_1 f(t_1) + \dots w_n f(t_n)].$$

Uma maneira direta de derivar fórmulas de integração é encontrar um interpolante e operar exatamente nele.

3.5.1 Regra do trapézio

Uma das fórmulas de integração mais importantes resulta da integração do interpolante linear por partes. Geometricamente, a fórmula trapezoidal de áreas de trapezoides que se aproximam da região sob a curva $y = f(x)$



Usando áreas de triângulos, é trivial derivar que

$$w_i = \begin{cases} 1, & i = 1, \dots, n-1, \\ \frac{1}{2}, & i = 0, n. \end{cases}$$

```

"""
    trapezoidal(f,a,b,n)

Aplicar a fórmula de integração do trapezoidal no intervalo [a, b], quebrado
em partes iguais. Retorna a estimativa, um vetor de nós e um vetor de valores do
integrando nos nós.
"""

function trapezoidal(f,a,b,n)
    h = (b-a)/n
    t = range(a,b,length=n+1)
    y = f.(t)
    T = h * ( sum(y[2:n]) + 0.5*(y[1] + y[n+1]) )
    return T,t,y
end

```

Vamos aproximar $f(x) = e^{\sin 7x}$ no intervalo $[0, 2]$.

```

using PrettyTables
f = x -> exp(sin(7*x));
a = 0; b = 2;

Integral = 2.6632197827615394 #exato
T,t,y = trapezoidal(f,a,b,40)
@show (T,Integral-T);

n = [ 10^n for n in 1:5 ]
err = []
for n in n

```

```

T,t,y = trapezoidal(f,a,b,n)
push!(err,Integral-T)
end

pretty_table([n err],["n","erro"])

```

Cada aumento em um fator de 10 em n reduz o erro em um fator de cerca de 100, o que é consistente com a convergência de segunda ordem.

3.5.2 Quadratura de Gauss

Vamos considerar a fórmula de integração numérica genérica:

$$\int_{-1}^1 f(x)dx \approx \sum_{k=1}^n w_k f(t_k) = Q_n[f],$$

Como existem n nós e n pesos disponíveis para escolher, parece plausível esperar que a quadratura consiga representar exatamente os polinômios de grau $m = 2n - 1$, e essa intuição acaba sendo correta.

Vamos testar isso na integral:

$$\int_{-1}^1 \frac{1}{1+4x^2} dx = \arctan(2).$$

```

f = x->1/(1+4*x^2);
exato = atan(2);

n = 8:4:96
errT = zeros(size(n))
errG = zeros(size(n))
for (k,n) in enumerate(n)
    errT[k] = abs(exato - trapezoidal(f,-1,1,n)[1])
    x, w = gausslegendre(n)
    errG[k] = abs(exato - dot(w, f.(x)))
end

errT[iszero.(errT)] .= NaN
errG[iszero.(errG)] .= NaN
plot(n,[errT errG],m=:o,label=["T" "GL"],
     xaxis=("nós"), yaxis=( :log10,"error",[1e-16,1]),
     title="integração numérica")

```

E agora com uma integral com um integrando mais pontudo:

$$\int_{-1}^1 \frac{1}{1+16x^2} dx = \frac{1}{2} \arctan(4).$$

```

f = x->1/(1+16*x^2);
exato = atan(4)/2;

n = 8:4:96
errT = zeros(size(n))

```

```

errG = zeros(size(n))
for (k,n) in enumerate(n)
    errT[k] = abs(exato - trapezoidal(f,-1,1,n)[1])
    x, w = gausslegendre(n)
    errG[k] = abs(exato - dot(w, f.(x)))
end

errT[iszero.(errT)] .= NaN
errG[iszero.(errG)] .= NaN
plot(n,[errT errG],m=:o,label=["T" "GL"],
    xaxis=("nós"), yaxis=( :log10,"error",[1e-16,1]),
    title="integração numérica")

```

3.5.3 Exercícios

1. Usando a mudança de variável: $z = \phi(x) = a + (b-a)\frac{(x+1)}{2}$ a integral pode ser reescrita como: $\int_a^b f(z) dz = \frac{b-a}{2} \int_{-1}^1 f(\phi(x)) dx$. Sabendo disso, use a quadratura de Gauss para integrar $\int_{\pi/2}^{\pi} x^2 \sin 8x dx = -\frac{3\pi^2}{32}$. Mostre resultados para diferentes valores de n até que se obtenha uma convergência de 10 dígitos.
2. Integre numericamente a função $f(x) = -x \log(|x - 1/2|)$ no intervalo $[0, 1/2]$ usando a quadratura de Gauss. Use 5, 10, 15 e 20 pontos de Gauss.

$$I_a = \int_0^{1/2} -x \log\left(\left|x - \frac{1}{2}\right|\right) dx = \frac{1}{16}(3 + 2 \log 2) = 0,274143$$

3.6 Integrais singulares ou quasi-singulares

A última função integrada tem uma singularidade quando $x = 1/2$. Mesmo com essa singularidade, e tendendo a infinito nesse ponto, essa função pode ser integrada analiticamente. Numericamente é interessante usar técnicas especiais para tratar de integrais desses tipo.

Considere uma integral do tipo:

$$I_s = \int_{-1}^1 \frac{g(\xi)}{(\xi - a)^2 + b^2} d\xi$$

Uma ideia para avaliar numericamente integrais singulares ($b = 0$) ou quasi-singulares ($b \neq 0$) é encontrar uma transformação cujo Jacobiano seja zero ou bem próximo de zero no ponto singular. Por exemplo, para integrais quasi-singulares podemos usar:

$$\xi = a + b \sinh(\mu s - \eta)$$

onde

$$\mu = \frac{1}{2} \left\{ \operatorname{arcsinh} \left(\frac{1+a}{b} \right) + \operatorname{arcsinh} \left(\frac{1-a}{b} \right) \right\}$$

$$\eta = \frac{1}{2} \left\{ \operatorname{arcsinh} \left(\frac{1+a}{b} \right) - \operatorname{arcsinh} \left(\frac{1-a}{b} \right) \right\}$$

e seu jacobiano é dado por:

$$\frac{d\xi}{ds} = b\mu \cosh(\mu s - \eta)$$

Aplicando essa transformação, obtém-se:

$$I_s = \int_{-1}^1 \frac{g(s(\xi))}{(s(\xi) - a)^2 + b^2} \frac{d\xi}{ds} ds$$

essa transformação suaviza o integrando e acelera a convergência da quadratura.

```
function sinhtrans(u, a, b)
    mu = 1 / 2 * (asinh((1 + a) / b) + asinh((1 - a) / b))
    eta = 1 / 2 * (asinh((1 + a) / b) - asinh((1 - a) / b))

    x = a .+ b * sinh.(mu * u .- eta)
    J = b * mu * cosh.(mu * u .- eta)
    x, J
end
```

```
x=range(-1,1,length=100)
xt,jac=sinhtrans(x, 0.5, 0.01)
plot(xt,[x,jac],layout=(2,1),xlabel=L"s",ylabel=[L"\xi" L"\frac{d\xi}{ds}"],legend=false)
```

aplicando essa técnica para a integral do último exemplo pode-se observar uma redução significativa do erro para a mesma quantidade de pontos.

```
f = x->1/(1+16*x^2);
exato = atan(4)/2;

n = 8

x, w = gausslegendre(n)
xt,J= sinhtrans(x, 0, 1/4)
errG = abs(exato - dot(w, f.(x)))
errGt = abs(exato - dot(w, J.*f.(xt)))
```

3.6.1 Exercício

1. Integre, outra vez , numericamente a função $f(x) = -x \log(|x - 1/2|)$ no intervalo $[0, 1/2]$ usando a quadratura de Gauss mas agora usando a transformada de Monegato com graus 3, 4 e 5. Use 5, 10, 15 e 20 pontos de Gauss e compare com os resultados.

```
"""
transformada de Monegato de grau q
singularidade em s0
"""
function Monegato(t, s0, q=5.0)
    delta = 2^(-q) * ((1 + s0)^(1 / q) + (1 - s0)^(1 / q))^q
    t0 = ((1 + s0)^(1 / q) - (1 - s0)^(1 / q)) / ((1 + s0)^(1 / q) + (1 - s0)^(1 / q))
    s = s0 .+ delta * (t .- t0) .^ q
    ds = q * delta * (t .- t0) .^ (q - 1)
    s, ds
end
```

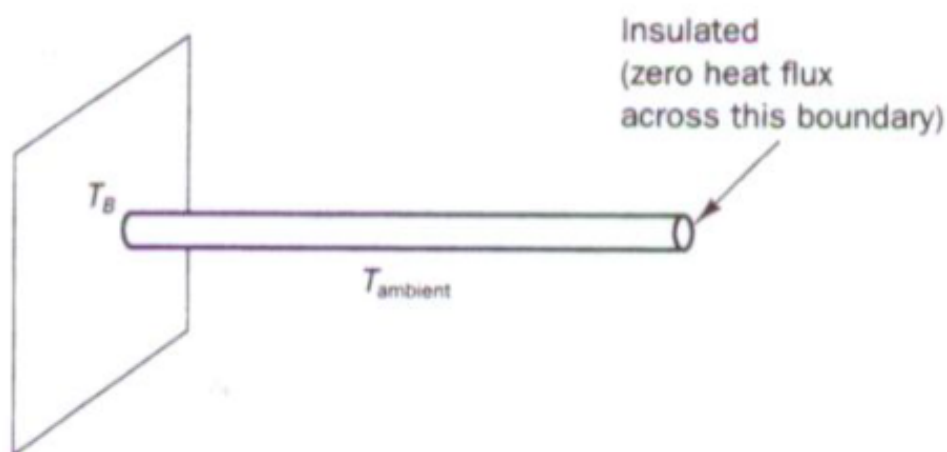
1. Use 10 pontos de Gauss para calcular as integrais abaixo. Compare com a solução analítica. Quais integrais tiveram os maiores erros? Por que?

$$= -0.666666666666667; \int_0^{\pi} (\sin(3x) \cdot \cos(2x) + 2x^3 + 3x \sin(x)) dx, 59.3293234777059; \int_0^1 \frac{1}{x+1} dx, \ln(2) = 0.6931471805599$$

3.7 Desafio

Considere o resfriamento de uma aleta circular por meio de transferência de calor por convecção ao longo de seu comprimento. A convecção dá origem a uma perda de calor ou termo de sumidouro dependente da temperatura na equação governante. Mostrada na Figura está uma aleta cilíndrica com área de seção transversal uniforme A . A base está a uma temperatura de 100°C (T_B) e a extremidade direita está isolada. A aleta está exposta a uma temperatura ambiente de 20°C . A transferência de calor unidimensional nesta situação é governada por $\frac{d}{dx} \left(kA \frac{dT}{dx} \right) - hP(T - T_\infty) = 0$

onde h é o coeficiente de transferência de calor por convecção, P o perímetro, k a condutividade térmica do material e T_∞ a temperatura ambiente. Calcule usando BEM a distribuição de temperatura ao longo da aleta e compare os resultados com a solução analítica fornecida por $\frac{T - T_\infty}{T_B - T_\infty} = \frac{\cosh[n(L-x)]}{\cosh(nL)}$



4 Equações diferenciais

Quantidades que mudam continuamente no tempo ou no espaço são frequentemente modeladas por equações diferenciais. As equações diferenciais precisam de condições suplementares para definir de maneira única tanto a situação de modelagem quanto as soluções teóricas. O problema de valor inicial (PVI), no qual todas as condições são dadas em um único valor da variável independente, é a situação mais simples. Muitas vezes, a variável independente neste caso representa o tempo.

Os métodos para PVIs geralmente começam a partir do valor inicial conhecido e iteram ou “avançam” a partir daí. Há um grande número desses métodos, em parte devido às diferenças em precisão, estabilidade e conveniência.

Um problema de valor inicial escalar de **primeira ordem** (PVI) é

$$u'(t) = f(t, u(t)), \quad a \leq t \leq b,$$

$$u(a) = u_0.$$

Chamamos t de **variável independente** e u de **variável dependente**. Se $u' = f(t, u) = g(t) + uh(t)$, a equação diferencial é **linear**; caso contrário, é **não linear**.

Uma **solução** de um problema de valor inicial é uma função $u(t)$ que torna ambas as equações $u'(t) = f(t, u(t))$ e $u(a) = u_0$ verdadeiras.

Quando t representa o tempo, às vezes escrevemos $\dot{u}$ (lê-se “u-ponto”) em vez de u' .

4.0.1 Soluções numéricas

O pacote `DifferentialEquations` oferece solucionadores para problemas de valor inicial (PVI). Vamos usá-lo para definir e resolver um problema de valor inicial para $u' = \sin[(u + t)^2]$ sobre $t \in [0, 4]$, tal que $u(0) = 1$.

Como muitos problemas práticos vêm com parâmetros que são fixos dentro de uma instância, mas variam de uma instância para outra, a sintaxe para PVI inclui um argumento de entrada p que permanece fixo durante toda a solução. Aqui não queremos usar esse argumento, mas ele deve estar na definição para que o solucionador funcione.

Para criar um problema de valor inicial para $u(t)$, você deve fornecer uma função que calcula u' , um valor inicial para u e os pontos finais do intervalo para t . O intervalo t deve ser definido como (a, b) , onde pelo menos um dos valores é um float.

```
using DifferentialEquations, LaTeXStrings
f = (u,p,t) -> sin((t+u)^2)      # define du/dt, deve incluir o argumento p
u0 = 1.0                        # valor inicial
tspan = (0.0,4.0)               # intervalo t
```

Com os dados acima, definimos um objeto de problema de PVI e depois o resolvemos. Aqui, informamos ao solucionador para usar o método `Tsit5`, que é uma boa escolha inicial para a maioria dos problemas.

```
ivp = ODEProblem(f,u0,tspan)
sol = solve(ivp,Tsit5());
```

O objeto solução resultante pode ser mostrado usando `plot`.

```
plot(sol,label="solução",legend=:bottom,
      xlabel="t",ylabel=L"u(t)",title=L"u'=\sin((t+u)^2)")
```

A solução também funciona como qualquer função que pode ser avaliada em diferentes valores de t .

```
@show sol(1.0);
```

Nos bastidores, o objeto solução contém algumas informações sobre como os valores e o gráfico são produzidos:

```
[sol.t sol.u]
```

O solucionador inicialmente encontra valores aproximados da solução (segunda coluna acima) em alguns tempos escolhidos automaticamente (primeira coluna acima). Para calcular a solução em outros momentos, o objeto realiza uma interpolação nesses valores. Este capítulo trata de como os valores

discretos de t e u são calculados. Por enquanto, apenas observe como podemos extraí-los do objeto solução.

```
scatter(sol.t,sol.u,label="valores discretos")
```

4.0.2 Método de Euler

Considere um problema de valor inicial de primeira ordem. Representamos uma solução numérica de um PVI por seus valores em uma coleção finita de nós, que por enquanto exigimos que sejam igualmente espaçados:

$$t_i = a + ih, \quad h = \frac{b-a}{n}, \quad i = 0, \dots, n.$$

h é chamado de **tamanho do passo**.

Como não obtemos valores exatamente corretos da solução nos nós, precisamos ter algum cuidado com a notação. A partir de agora, deixamos $\hat{u}(t)$ denotar a solução exata do PVI. O valor aproximado em t_i calculado por nossos métodos numéricos será denotado por $u_i \approx \hat{u}(t_i)$.

Considere um interpolador linear por partes para os valores (ainda desconhecidos) $u_0, u_1, \dots, u_n$. Para $t_i < t < t_{i+1}$, sua inclinação é

$$\frac{u_{i+1} - u_i}{t_{i+1} - t_i} = \frac{u_{i+1} - u_i}{h}.$$

Podemos conectar essa derivada à equação diferencial seguindo o modelo de $u' = f(t, u)$:

$$\frac{u_{i+1} - u_i}{h} = f(t_i, u_i), \quad i = 0, \dots, n-1.$$

Podemos ver o lado esquerdo como uma aproximação para $u'(t)$ em $t = t_i$. Podemos reorganizar a equação para obter o **método de Euler**, nosso primeiro método para PVIs.

O método de Euler avança em t , obtendo a solução em um novo nível de tempo explicitamente em termos do valor mais recente $u_{i+1} = u_i + hf(t_i, u_i)$.

```
"""
    euler(ivp,n)

    Aplique o método de Euler para resolver o PVI dado usando `n` passos de tempo.
    Retorna um vetor de tempos e um vetor de valores da solução.
"""
function euler(ivp,n)
    # Discretização do tempo.
    a,b = ivp.tspan
    h = (b-a)/n
    t = [ a + i*h for i in 0:n ]

    # Condição inicial e configuração de saída.
    u = fill(float(ivp.u0),n+1)

    # A iteração de passos de tempo.
    for i in 1:n
        u[i+1] = u[i] + h*ivp.f(u[i],ivp.p,t[i])
    end
```

```

    return t,u
end

```

4.0.3 Exemplo

Consideramos o PVI $u' = \sin[(u + t)^2]$ sobre $0 \leq t \leq 4$, com $u(0) = -1$.

```

f = (u,p,t) -> sin((t+u)^2);
tspan = (0.0,4.0);
u0 = -1.0;

ivp = ODEProblem(f,u0,tspan)
t,u = euler(ivp,20)

plot(t,u,m=2,label="n=20",
      xlabel=L"t",ylabel=L"u(t)",title="Solução por Euler" )

```

Poderíamos definir um interpolador diferente para obter uma imagem mais suave acima, mas a derivação do método de Euler assumiu um interpolador linear por partes. Podemos, em vez disso, solicitar mais passos para fazer o interpolador parecer mais suave.

```

t,u = euler(ivp,50)
plot!(t,u,m=2,label="n=50")

```

Aumentar n mudou a solução de maneira notável. Como sabemos que interpoladores e diferenças finitas se tornam mais precisos à medida que $h \rightarrow 0$, devemos antecipar o mesmo comportamento do método de Euler. Não temos uma solução exata para comparar, então usaremos um solucionador DifferentialEquations para construir uma solução de referência precisa.

```

u_exact = solve(ivp,Tsit5(),reltol=1e-14, abstol=1e-14)

plot!(u_exact,l=(2,:black),label="referência")

```

Agora podemos realizar um estudo de convergência.

```

n = [ round{Int,5*10^k} for k in 0:0.5:3 ]
err = []
for n in n
    t,u = euler(ivp,n)
    push!( err, norm(u_exact.(t)-u,Inf) )
end

pretty_table((n=n,err=err),header=["n","Inf-norm error"])

```

O erro é aproximadamente reduzido por um fator de 10 para cada aumento em n pelo mesmo fator.

4.1 Sistemas de equações

Poucas aplicações envolvem um problema de valor inicial com apenas uma única variável dependente. Geralmente, existem múltiplas incógnitas e um sistema de equações para defini-las.

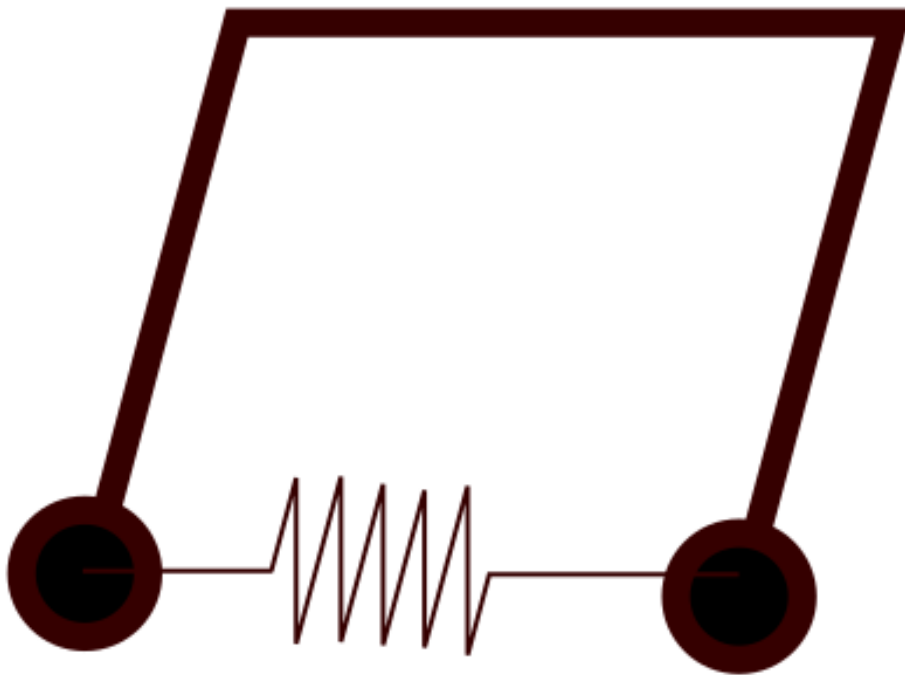
A generalização de qualquer solucionador escalar de PVI para lidar com sistemas é direta. Considere o método de Euler, que na forma de sistema se torna

$$\mathbf{u}_{i+1} = \mathbf{u}_i + h \mathbf{f}(t_i, \mathbf{u}_i), \quad i = 0, \dots, n-1.$$

A equação de diferenças vetoriais é apenas a fórmula de Euler aplicada simultaneamente a cada componente do sistema de EDO. Como operações como adição e multiplicação se traduzem facilmente de escalares para vetores, a função que escrevemos para PVIs escalares funciona para sistemas também. Praticamente falando, as únicas mudanças que devem ser feitas são que a condição inicial e a função de EDO têm que ser codificadas para usar vetores.

Felizmente, a capacidade de resolver sistemas de de primeira ordem implica também a capacidade de resolver sistemas de ordem diferencial mais alta. A razão é que existe uma maneira sistemática de transformar um problema de ordem superior em um de primeira ordem de dimensão superior.

Dois pêndulos idênticos suspensos na mesma barra e oscilando em planos paralelos podem ser modelados como o sistema de segunda ordem



$$\begin{aligned} \theta_1''(t) + \gamma \theta_1' + \frac{g}{L} \sin \theta_1 + k(\theta_1 - \theta_2) &= 0, \\ \theta_2''(t) + \gamma \theta_2' + \frac{g}{L} \sin \theta_2 + k(\theta_2 - \theta_1) &= 0, \end{aligned}$$

onde θ_1 e θ_2 são os ângulos feitos pelos dois pêndulos, L é o comprimento de cada pêndulo, γ é um parâmetro de fricção, e k é um parâmetro que descreve um torque produzido pela barra quando ela é torcida. Podemos converter este problema em um sistema de primeira ordem usando as substituições

$$u_1 = \theta_1, \quad u_2 = \theta_2, \quad u_3 = \theta_1', \quad u_4 = \theta_2'$$

Com essas definições, o sistema se torna

$$\begin{aligned}
u_1' &= u_3, \\
u_2' &= u_4, \\
u_3' &= -\gamma u_3 - \frac{g}{L} \sin u_1 + k(u_2 - u_1), \\
u_4' &= -\gamma u_4 - \frac{g}{L} \sin u_2 + k(u_1 - u_2),
\end{aligned}$$

que é um sistema de primeira ordem em quatro dimensões. Para completar a descrição do problema, é necessário especificar valores para $\theta_1(0)$, $\theta_1'(0)$, $\theta_2(0)$ e $\theta_2'(0)$.

O truque ilustrado nos exemplos anteriores está sempre disponível. Para cada variável dependente escalar no sistema, introduza novos componentes até a derivada mais alta que aparece para y . As equações do sistema de primeira ordem vêm das relações triviais entre todas as derivadas inferiores e das equações originais de alta ordem. No final, deve haver tantas equações de componentes escalares quanto variáveis desconhecidas de primeira ordem.

4.1.1 Exemplo

```
function couple(u,p,t)
    gamma,L,k = p
    g = 9.8
    udot = similar(u)
    udot[1:2] .= u[3:4]
    udot[3] = - gamma*u[3] - (g/L)*sin(u[1]) + k*(u[2]-u[1])
    udot[4] = - gamma*u[4] - (g/L)*sin(u[2]) + k*(u[1]-u[2])
    return udot
end
u0 = [1.25,-0.5,0,0]
tspan = (0.,50.);
gamma,L,k = 0,0.5,0
ivp = ODEProblem(couple,u0,tspan,[gamma,L,k])
sol = solve(ivp,Tsit5())
plot(sol,vars=[1,2],label=[L"\theta_1" L"\theta_2"],
      xlims=[20,50],title="k=0")

k = 1
ivp = ODEProblem(couple,u0,tspan,[gamma,L,k])
sol = solve(ivp,Tsit5())
plot(sol,vars=[1,2],label=[L"\theta_1" L"\theta_2"],
      xlims=[20,50],title="k=1")
```

4.2 Runge-Kutta

Chegamos agora a um dos principais e mais utilizados tipos de métodos para problemas de valor inicial: métodos Runge-Kutta. São métodos de uma etapa, embora eles não sejam frequentemente escritos nessa forma. Os métodos RK aumentam a precisão da primeira ordem, avaliando a função da EDO $f(t, u)$ mais de uma vez por etapa do tempo.

- Método de segunda ordem Considere uma expansão em série da solução exata para $u' = f(t, u)$,

$$\hat{u}(t_{i+1}) = \hat{u}(t_i) + h\hat{u}'(t_i) + \frac{1}{2}h^2\hat{u}''(t_i) + O(h^3).$$

Se substituirmos $\hat{u}'$ por f e mantivermos apenas os dois primeiros termos no lado direito, obteremos o método de Euler. Para obter mais precisão, precisaremos calcular ou estimar o terceiro termo também. Observe que

$$\hat{u}'' = f' = \frac{df}{dt} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial u} \frac{du}{dt} = f_t + f_u f,$$

onde aplicamos a regra da cadeia multidimensional à derivada, porque ambos os argumentos de f dependem de t . Usando essa expressão, obtemos

$$\begin{aligned} \hat{u}(t_{i+1}) = \hat{u}(t_i) + h \left[f(t_i, \hat{u}(t_i)) + \frac{h}{2} f_t(t_i, \hat{u}(t_i)) + \frac{h}{2} f(t_i, \hat{u}(t_i)) f_u(t_i, \hat{u}(t_i)) \right] \\ + O(h^3). \end{aligned}$$

Uma aproximação dessas derivadas parciais de f é necessária. Observe que

$$f(t_i + \alpha, \hat{u}(t_i) + \beta) = f(t_i, \hat{u}(t_i)) + \alpha f_t(t_i, \hat{u}(t_i)) + \beta f_u(t_i, \hat{u}(t_i)) + O(\alpha^2 + |\alpha\beta| + \beta^2).$$

Correspondendo esta expressão ao termo entre colchetes e selecionando $\alpha = h/2$ e $\beta = \frac{1}{2}hf(t_i, \hat{u}(t_i))$. Fazendo isso, encontramos

$$\hat{u}(t_{i+1}) = \hat{u}(t_i) + h[f(t_i + \alpha, \hat{u}(t_i) + \beta)] + O(h\alpha^2 + h|\alpha\beta| + h\beta^2 + h^3).$$

Truncar a série aqui resulta em um novo método de uma etapa.

4.2.1 Método de Euler Melhorado

O método de Euler melhorado é a fórmula de uma etapa

$$u_{i+1} = u_i + hf\left(t_i + \frac{1}{2}h, u_i + \frac{1}{2}hf(t_i, u_i)\right).$$

Graças às definições acima a ordem de precisão do Euler melhorado é dois. Em um primeiro estágio, o método faz meio passo de Euler $h/2$

$$k_1 = hf(t_i, u_i),$$

$$v = u_i + \frac{1}{2}k_1.$$

e no segundo estágio ele usa o passo inteiro de Euler mas usa o valor obtido no primeiro estágio para a inclinação.

$$k_2 = hf\left(t_i + \frac{1}{2}h, v\right),$$

$$u_{i+1} = u_i + k_2.$$

```
"""
```

```
    euler2(ivp,n)
```

```
Aplique o método de Euler Melhorado para resolver o PVI dado usando `n`
passos de tempo. Retorna um vetor de tempos e um vetor de valores da solução.
```

```
"""
```

```
function me2(ivp,n)
    # Discretização do tempo.
    a,b = ivp.tspan
    h = (b-a)/n
```

```

t = [ a + i*h for i in 0:n ]

# Inicializar saída.
u = fill(float(ivp.u0),n+1)

# Iteração de passos de tempo.
for i in 1:n
    uhalf = u[i] + h/2*ivp.f(u[i],ivp.p,t[i]);
    u[i+1] = u[i] + h*ivp.f(uhalf,ivp.p,t[i]+h/2);
end
return t,u
end

```

Esse procedimento pode ser feito para ordens mais altas mas a complexidade aumenta rapidamente.

O método de Runge-Kutta mais comumente usado, e talvez o método mais popular de todos, é o de quarta ordem, dado por:

$$\begin{aligned}
 k_1 &= hf(t_i, u_i), \\
 k_2 &= hf(t_i + h/2, u_i + k_1/2), \\
 k_3 &= hf(t_i + h/2, u_i + k_2/2), \\
 k_4 &= hf(t_i + h, u_i + k_3), \\
 u_{i+1} &= u_i + \frac{1}{6}k_1 + \frac{1}{3}k_2 + \frac{1}{3}k_3 + \frac{1}{6}k_4.
 \end{aligned}$$

```

"""
    rk4(ivp,n)

Aplicar o método comum de Runge-Kutta de 4ª ordem para resolver o PVI dado
usando `n` passos de tempo. Retorna um vetor de tempos e um vetor de
valores da solução.
"""
function rk4(ivp,n)
    # Discretização do tempo.
    a,b = ivp.tspan
    h = (b-a)/n
    t = [ a + i*h for i in 0:n ]

    # Inicializar saída.
    u = fill(float(ivp.u0),n+1)

    # Iteração de passos de tempo.
    for i in 1:n
        k1 = h*ivp.f( u[i],      ivp.p, t[i]      )
        k2 = h*ivp.f( u[i]+k1/2, ivp.p, t[i]+h/2 )
        k3 = h*ivp.f( u[i]+k2/2, ivp.p, t[i]+h/2 )
        k4 = h*ivp.f( u[i]+k3,   ivp.p, t[i]+h   )
        u[i+1] = u[i] + (k1 + 2(k2+k3) + k4)/6
    end
    return t,u
end

```

```
f = (u,p,t) -> sin((t+u)^2)
ivp = ODEProblem(f,u0,tspan)
tspan = (0.0,4.0)
u0 = -1.0
u_ref = solve(ivp,Tsit5(),reltol=1e-14, abstol=1e-14);

n = [ round(Int,2*10^k) for k in 0:0.5:3 ]
err_IE2,err_RK4 = [],[]
for n in n
    t,u = euler2(ivp,n)
    push!( err_IE2, maximum( @.abs(u_ref(t)-u) ) )
    t,u = rk4(ivp,n)
    push!( err_RK4, maximum( @.abs(u_ref(t)-u) ) )
end

pretty_table([n err_IE2 err_RK4],header=["n","IE2 erro","RK4 erro"])
```

4.3 Métodos de múltiplos passos

Nos métodos de Runge–Kutta, começamos em u_i para encontrar u_{i+1} , realizando múltiplas avaliações de f (estágios) para alcançar alta precisão. Em contraste, os métodos de múltiplos passos aumentam a precisão utilizando mais do histórico da solução, aproveitando informações do passado recente. Para a discussão nesta e nas seções seguintes, introduzimos a notação abreviada $f_i = f(t_i, u_i)$.

Um método de **múltiplos passos** (ou *método linear de múltiplos passos*) de k passos é dado pela

$$u_{i+1} = a_{k-1}u_i + \dots + a_0u_{i-k+1} + h(b_k f_{i+1} + \dots + b_0 f_{i-k+1}),$$

onde a_j e b_j são constantes. Se $b_k = 0$, o método é **explícito**; caso contrário, é **implícito**.

As quantidades u e f são mostradas como escalares, mas em geral podem ser vetores.

Para usar como um método numérico, iteramos através de $i = k - 1, \dots, n - 1$. O valor u_0 é determinado pela condição inicial, mas também precisamos de alguma forma de gerar os **valores iniciais** $u_1 = \alpha_1, \dots, u_{k-1} = \alpha_{k-1}$.

A fórmula define u_{i+1} em termos de valores conhecidos da solução e sua derivada do passado. No caso explícito com $b_k = 0$, a Equação imediatamente fornece uma fórmula para a quantidade desconhecida u_{i+1} em termos de valores no nível de tempo t_i e anteriores. Assim, apenas uma nova avaliação de f é necessária para fazer um passo de tempo, desde que armazenemos o histórico recente.

Por exemplo a formula do Adams-Bashforth de quarta ordem é dada por:

$$\mathbf{u}_{i+1} = \mathbf{u}_i + h \left(\frac{55}{24}\mathbf{f}_i - \frac{59}{24}\mathbf{f}_{i-1} + \frac{37}{24}\mathbf{f}_{i-2} - \frac{9}{24}\mathbf{f}_{i-3} \right), \quad i = 3, \dots, n-1.$$

```
function ab4(ivp,n)
    # Discretização do tempo.
    a,b = ivp.tspan
    h = (b-a)/n
    t = [ a + i*h for i in 0:n ]

    # Constantes no método AB4.
    k = 4;    σ = [-9,37,-59,55]/24;

    # Encontrar valores iniciais usando RK4.
```

```

u = fill(float(ivp.u0),n+1)
rkivp = ODEProblem(ivp.f,ivp.u0,(a,a+(k-1)*h),ivp.p)
ts,us = rk4(rkivp,k-1)
u[1:k] .= us

# Calcular histórico dos valores de u', do mais recente ao mais antigo.
f = [ ivp.f(u[i],ivp.p,t[i]) for i in 1:k ]
# Iteração de passos de tempo.
for i in k+1:n+1
    u[i] = u[i-1] + h*dot(f,σ) # avançar um passo
    f = [ f[2:k];ivp.f(u[i],ivp.p,t[i]) ] # novo valor de du/dt
end
return t,u
end

```

Agora fazemos um estudo de convergência:

```

err_AB4 = []
for n in n
    t,u = ab4(ivp,n)
    push!( err_AB4, maximum( @.abs(u_ref(t)-u) ) )
end

pretty_table([n err_IE2 err_RK4 err_AB4],["n","IE2 erro","RK4 erro","AB4 erro"])

```

Os métodos de Adams-Moulton são métodos implícitos e, portanto, não podem ser resolvidos como fazemos no caso dos métodos de Adams-Bashforth. Portanto, para obter o valor aproximado de u_{i+1} , podemos usar um método de dois passos chamado método preditor-corretor.

No passo 1, usamos um método explícito, como o método de Adams-Bashforth, chamado preditor, para obter um valor aproximado inicial e no passo 2, chamado corretor, melhoramos a estimativa inicial.

O método Adams-Moulton implícito de quarta ordem é dado por:

$$\mathbf{u}_{i+1}^k = \mathbf{u}_i + h \left(\frac{9}{24}\mathbf{f}_{i+1}^{k-1} + \frac{19}{24}\mathbf{f}_i - \frac{5}{24}\mathbf{f}_{i-1} + \frac{1}{24}\mathbf{f}_{i-2} \right)$$

esse passo pode ser repetido até obter $|u_{i+1}^{(k)} - u_{i+1}^{(k-1)}| \leq \epsilon |u_{i+1}^{(k-1)}|$

```

function am4(ivp, n; tol=1e-8, max_iter=10)
    # Discretização do tempo.
    a, b = ivp.tspan
    h = (b - a) / n
    t = [a + i * h for i in 0:n]

    # Constantes do método AM4.
    k = 4
    σ = [1, -5, 19, 9] / 24
    σb = [-9, 37, -59, 55] / 24

    # Encontrar valores iniciais usando RK4.
    u = fill(float(ivp.u0), n + 1)
    rkivp = ODEProblem(ivp.f, ivp.u0, (a, a + (k - 1) * h), ivp.p)
    ts, us = rk4(rkivp, k - 1)
    u[1:k] .= us

```

```

# Calcular histórico dos valores de u', do mais recente ao mais antigo.
f = [ ivp.f(u[i],ivp.p,t[i]) for i in 1:k ]
# Iteração de passos de tempo.
for i in k+1:n+1
    u[i] = u[i-1] + h*dot(f,ob) # avançar um passo
    f = [ f[2:k];ivp.f(u[i],ivp.p,t[i]) ] # novo valor de du/dt

    # Método de correção.
    u_corr=0
    for iter in 1:max_iter
        u_corr = u[i-1] + h*dot(f,o) # avançar um passo
        # @show iter,abs(u[i] - u_corr) , tol * abs(u_corr)
        if abs(u[i] - u_corr) <= tol * abs(u_corr)
            break
        end
        f[4] = ivp.f(u_corr,ivp.p,t[i]) # novo valor de du/dt
        u[i] = u_corr
    end
end

return t, u
end

```

4.3.1 Exercícios - Passo no tempo

1- Escolha 5 PVI, use as funções rk4 ,ab4 e am4 e resolva para $n = 10 \cdot 2^d$ e $d = 1, \dots, 10$. Faça um gráfico de convergência log-log para os erros no tempo final $|u_n - \hat{u}(t_n)|$ por n , e adicione uma linha reta indicando a convergência de quarta ordem. Com $n = 100$, trace a solução e o erro $u - \hat{u}$ em gráficos separados.

$$u' = -2tu,$$

$$0 \leq t \leq 2,$$

$$u(0) = 2;$$

$$\hat{u}(t) = 2e^{-t^2}$$

$$u' = u + t,$$

$$0 \leq t \leq 1,$$

$$u(0) = 2;$$

$$\hat{u}(t) = 1 - t + e^t$$

$$u' = x^2/[u(1+x^3)],$$

$$0 \leq x \leq 3,$$

$$u(0) = 1;$$

$$\hat{u}(x) = [1 + (2/3) \ln(1+x^3)]^{1/2}$$

$$u'' + 9u = 9t,$$

$$0 < t < 2\pi,$$

$$u(0) = 1,$$

$$u'(0) = 1;$$

$$\hat{u}(t) = t + \cos(3t)$$

$$u'' + 9u = \sin(2t),$$

$$0 < t < 2\pi,$$

$$u(0) = 2,$$

$$u'(0) = 1; \quad \hat{u}(t) = (1/5) \sin(3t) + 2 \cos(3t) + (1/5) \sin(2t)$$

$$u'' - 9u = 9t$$

$$0 < t < 1,$$

$$u(0) = 2,$$

$$u'(0) = -1;$$

$$\hat{u}(t) = e^{3t} + e^{-3t} - t$$

$$u'' + 4u' + 4u = t,$$

$$0 < t < 4,$$

$$u(0) = 1,$$

$$u'(0) = 3/4;$$

$$\hat{u}(t) = (3t + 5/4)e^{-2t} +$$

$$x^2u'' + 5xu' + 4u = 0,$$

$$1 < x < e^2,$$

$$u(1) = 1,$$

$$u'(1) = -1;$$

$$\hat{u}(x) = x^{-2}(1 + \ln x)$$

$$2x^2u'' + 3xu' - u = 0,$$

$$1 < x < 16,$$

$$u(1) = 4,$$

$$u'(1) = -1; \quad \hat{u}(x) = 2(x^{1/2} + x^{-1})$$

$$x^2u'' - xu' + 2u = 0,$$

$$1 < x < e^\pi,$$

$$u(1) = 3,$$

2- O método de Houbolt é comumente usado em problemas de dinâmica estrutural. O método é conhecido por sua estabilidade e é especialmente eficaz em problemas onde é necessário lidar com altas frequências ou amortecimento. Ele descreve as derivadas de ordem 1 e 2 como:

$$u''_{n+1} = \frac{2u_{n+1} - 5u_n + 4u_{n-1} - u_{n-2}}{h^2}$$

$$u'_{n+1} = \frac{11u_{n+1} - 18u_n + 9u_{n-1} - 2u_{n-2}}{6h}$$

implemente esse método e o compare com um dos problemas resolvidos na questão anterior.

4.4 Matrizes de diferenças finitas

Primeiro discretizamos o intervalo $x \in [a, b]$ em pedaços iguais de comprimento $h = \frac{b-a}{n}$, levando aos nós $x_i = a + ih$, $i = 0, \dots, n$.

Nosso objetivo é encontrar um vetor $\mathbf{g}$ tal que $g_i \approx f'(x_i)$ para $i = 0, \dots, n$. Usando a fórmula de diferenças finitas:

$$g_n = \frac{f_n - f_{n-1}}{h}.$$

Podemos resumir todo o conjunto de fórmulas definindo

$$\mathbf{f} = \begin{bmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_{n-1}) \\ f(x_n) \end{bmatrix},$$

e então a equação vetorial

$$\begin{bmatrix} f'(x_0) \\ [1mm]f'(x_1) \\ [1mm] \vdots \\ [1mm]f'(x_{n-1}) \\ [1mm]f'(x_n) \end{bmatrix} \approx \mathbf{D}_x \mathbf{f}, \quad \mathbf{D}_x = \frac{1}{h} \begin{bmatrix} -1 & 1 & & & \\ [1mm] & -1 & 1 & & \\ [1mm] & & \ddots & \ddots & \\ [1mm] & & & -1 & 1 \\ [1mm] & & & & -1 & 1 \end{bmatrix}.$$

Aqui, como em outros lugares, os elementos de $\mathbf{D}_x$ que não são mostrados são zero. Chamamos $\mathbf{D}_x$ de **matriz de diferenciação**. Cada linha de $\mathbf{D}_x$ fornece os pesos da fórmula de diferença finita usada em um dos nós.

A matriz de diferenciação não é uma escolha única. Somos livres para usar quaisquer fórmulas de diferença finita que quisermos em cada linha. No entanto, faz sentido escolher linhas que sejam o mais semelhantes possível. Usando diferenças centradas de segunda ordem onde possível e fórmulas unilaterais de segunda ordem nos pontos de fronteira resulta em

$$\mathbf{D}_x = \frac{1}{h} \begin{bmatrix} -\frac{3}{2} & 2 & -\frac{1}{2} & & & \\ [1mm] -\frac{1}{2} & 0 & \frac{1}{2} & & & \\ [1mm] & -\frac{1}{2} & 0 & \frac{1}{2} & & \\ & & \ddots & \ddots & \ddots & \\ & & & -\frac{1}{2} & 0 & \frac{1}{2} \\ [1mm] & & & \frac{1}{2} & -2 & \frac{3}{2} \end{bmatrix}.$$

As matrizes de diferenciação até agora são matrizes bandadas, ou seja, todos os valores não zero estão ao longo das diagonais próximas à diagonal principal.

4.4.1 Segunda derivada

Da mesma forma, podemos definir matrizes de diferenciação para segundas derivadas. Por exemplo,

$$\begin{bmatrix} f''(x_0) \\ [1mm] f''(x_1) \\ [1mm] f''(x_2) \\ [1mm] \vdots \\ [1mm] f''(x_{n-1}) \\ [1mm] f''(x_n) \end{bmatrix} \approx \frac{1}{h^2} \begin{bmatrix} 2 & -5 & 4 & -1 \\ [1mm] 1 & -2 & 1 & \\ [1mm] & 1 & -2 & 1 \\ [1mm] & & \ddots & \ddots & \ddots \\ [1mm] & & & 1 & -2 & 1 \\ [1mm] & & & & -1 & 4 & -5 & 2 \end{bmatrix} \begin{bmatrix} f(x_0) \\ [1mm] f(x_1) \\ [1mm] f(x_2) \\ [1mm] \vdots \\ [1mm] f(x_{n-1}) \\ [1mm] f(x_n) \end{bmatrix} = \mathbf{D}_{xx} \mathbf{f}.$$

```

"""
    diff(n, xspan)

Construa matrizes de diferenciação de 2ª ordem, usando `n` nós únicos no
intervalo
`xspan`. Retorna um vetor de nós e as matrizes para as primeiras
e segundas derivadas.
"""
function diffmat(n, xspan)
    a,b = xspan
    h = (b-a)/n
    x = [ a + i*h for i in 0:n ]    # nós

    # Define a maior parte de Dx por suas diagonais.
    dp = fill(0.5/h,n)              # superdiagonal
    dm = fill(-0.5/h,n)             # subdiagonal
    Dx = diagm(-1=>dm,1=>dp)

    # Corrigir as primeiras e últimas linhas.
    Dx[1,1:3] = [-1.5,2,-0.5]/h
    Dx[n+1,n-1:n+1] = [0.5,-2,1.5]/h

    # Define a maior parte de Dxx por suas diagonais.
    d0 = fill(-2/h^2,n+1)          # diagonal principal
    dp = ones(n)/h^2                # super- e subdiagonal
    Dxx = diagm(-1=>dp,0=>d0,1=>dp)

    # Corrigir as primeiras e últimas linhas.
    Dxx[1,1:4] = [2,-5,4,-1]/h^2
    Dxx[n+1,n-2:n+1] = [-1,4,-5,2]/h^2

    return x,Dx,Dxx
end

```

Usando essas matrizes para resolver o problema com condições de contorno

$$T(-1) = 100, T(1) = 0$$

```

n=100
x,dx,dxx=diffmat(n,[-1,1])
dxx[1,:].=0
dxx[end,:].=0
dxx[1,1]=1
dxx[end,end]=1
b=zeros(n+1)
b[1]=100
xdf=dxx\b

```

4.5 A equação de difusão

A equação de difusão em uma dimensão é

$$u_t = ku_{xx},$$

onde k é o *coeficiente de difusão*. A equação do calor é a equação diferencial típica para a classe conhecida como **EDPs parabólicas**. Um processo difusivo é aquele em que a velocidade é proporcional ao gradiente da solução. Assim, mudanças rápidas na solução se achatam rapidamente.

Agora vamos resolver a equação de difusão em $[-1, 1]$ usando diferenças finitas para aproximar a derivada em x . As condições de contorno são $u(-1, t) = 0, u(1, t) = 2$ e a condição inicial é $1 + \sin(\pi x/2) + 3(1 - x^2)e^{-4x^2}$.

```
using DifferentialEquations, LaTeXStrings, Plots

n=100
x,dx,dxx=diffmat(n, [-1,1])

f = (u,p,t) -> p[1]*[p[2];u[2:end-1];p[3]]
init = x -> 1 + sin(π*x/2) + 3*(1-x^2)*exp(-4x^2);
u0=init.(x)
tspan=(0,0.75)
ivp = ODEProblem(f,u0,tspan,[dxx,0,2])
sol = solve(ivp,Tsit5());

plt = plot(xlabel=L"x",ylabel=L"u(x,t)",legend=:topleft,
           title="Solução da equação de calor")
for t in 0:0.1:0.7
    plot!(x[1:end-1],sol(t)[1:end-1],label="t=$t")
end
plt

anim = @animate for t in range(0,0.75,length=201)
    plot(x[1:end-1],sol(t)[1:end-1],label="t=$t",
         xaxis=(L"x"), yaxis=(L"u(x,t)",(0,4.2)),
         title="Difusão",leg=:topleft,dpi=100)
end
mp4(anim,"calor.mp4",fps=30)
```

Vídeo: [boundaries-heat.mp4](#)

Aqui as estratégias de passo no tempo apresentadas anteriormente também poderiam ser utilizadas.

4.6 Exercício -BEM x MDF

1 - Usando o BEM resolva o mesmo problema de **difusão** e compare com o MDF. Como o termo transiente aparece na equação integral? O que precisa ser feito para descrever u_t em termos das matrizes do BEM?

4.7 Equação da onda

A equação da onda é dada por $u_{tt} - c^2 u_{xx} = 0$. Usaremos $x \in [0, 1]$ e $t > 0$ como o domínio.

Para reduzir a ordem desse problema podemos definir:

$$u_t = y,$$

$$y_t = c^2 u_{xx}.$$

o que resultaria no sistema matricial:

$$\begin{bmatrix} \mathbf{u}'(t) \\ [2mm]\mathbf{y}'(t) \end{bmatrix} = \begin{bmatrix} \mathbf{0} & I \\ [2mm]c^2\mathbf{D}_{xx} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{u}(t) \\ [2mm]\mathbf{y}(t) \end{bmatrix}.$$

Usaremos velocidade $c = 2$, as condições de Dirichlet $u(0, t) = u(1, t) = 0$ e duas condições iniciais:

$$u(x, 0) = e^{-100(x+0.5)^2}, \quad 0 \leq x \leq 1,$$

$$u_t(x, 0) = -u(x, 0), \quad 0 \leq x \leq 1.$$

```
n=100
x,dx,dxx=diffmat(n,[-1,1])

f = (u,p,t) -> p[1]*[p[2];u[2:p[4]-1];p[3];u[p[4]+1:end]]
init = x -> exp(-100*(x+0.5)^2);
u0=[init.(x); -init.(x)]
tspan=(0,2)
c=2
ivp = ODEProblem(f,u0,tspan,[[zeros(n+1,n+1) I;c^2*dxx zeros(n+1,n+1)],0,0,n+1])
sol = solve(ivp,Tsit5());

plt = plot(xlabel=L"x",ylabel=L"u(x,t)",legend=:topleft,
           title="Solução da equação da onda")
for t in 0:0.2:2
    plot!(x[1:n-1],sol(t)[1:n-1],label="t=$t")
end
plt

anim = @animate for t in range(0,2,length=201)
    plot(x[1:n-1],sol(t)[1:n-1],label="t=$t",
         xaxis=(L"x"), yaxis=(L"u(x,t)",(-1.1,1.1)),
         title="onda",leg=:topleft,dpi=100)
end
mp4(anim,"onda.mp4",fps=30)
```

Vídeo: onda.mp4

4.8 Desafio

Usando o BEM resolva o mesmo problema e compare com o MDF.

```
x0=0
xf=1
l=xf-x0
H=[0.5 -.5
  -.5 .5]
G=-[0 l/2
    -l/2 0]
#T=0.5*(x1[1]+x1[2]).+(xs.-x0)/2*x1[3].+(xs.-xf)/2*x1[4]

Hi=[0.5 0.5]
Gi=[(xs.-x0)/2 (xs.-xf)/2]

Ht=[H zeros(2,2);Hi -I]
Gt=[G ;Gi ]
```

5 Viga de Euler

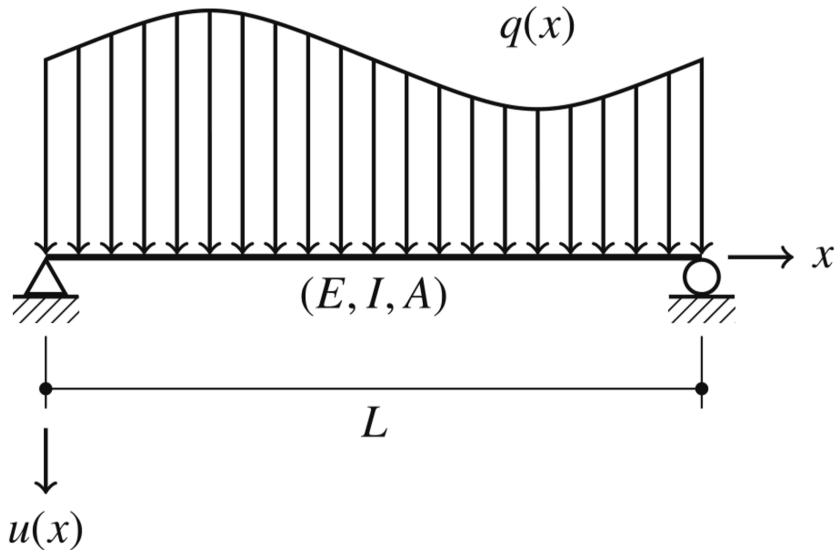
5.1 Teoria de vigas

Considerando a viga representada a equação governante do problema pode ser escrita como :

$$EI \frac{d^4 u}{dx^4} = q(x)$$

A partir de u as podemos definir:

$\varphi = \frac{du}{dx}$, $M = -EI \frac{d^2 u}{dx^2}$, e $Q = -EI \frac{d^3 u}{dx^3}$ que são a rotação, momento fletor e cortante.



usando essas definições, aplicando o método dos resíduos ponderados e integração por partes 4 vezes obtemos:

$$\begin{aligned} u(\xi) = & +u^*(\xi, x)Q(x) \big|_{x=L} - u^*(\xi, x)Q(x) \big|_{x=0} \\ & -\varphi^*(\xi, x)M(x) \big|_{x=L} + \varphi^*(\xi, x)M(x) \big|_{x=0} \\ & +M^*(\xi, x)\varphi(x) \big|_{x=L} - M^*(\xi, x)\varphi(x) \big|_{x=0} \\ & -Q^*(\xi, x)u(x) \big|_{x=L} + Q^*(\xi, x)u(x) \big|_{x=0} \\ & + \int_0^L u^*(\xi, x)q(x) dx, \end{aligned}$$

onde:

$$\begin{aligned} u^*(\xi, x) &= \frac{r^3}{12EI}, \\ \varphi^*(\xi, x) &= \frac{r^2}{4EI} \left(\frac{\partial r}{\partial x} \right), \\ M^*(\xi, x) &= -\frac{r}{2} \left(\frac{\partial r}{\partial x} \right)^2, \\ Q^*(\xi, x) &= -\frac{1}{2} \left(\frac{\partial r}{\partial x} \right)^3 \text{ e} \\ r &= |x - \xi| \end{aligned}$$

onde $\frac{\partial r}{\partial x} = -1$ se $x < \xi$ e $\frac{\partial r}{\partial x} = 1$ se $x > \xi$.

Para esse problema precisamos de definir mais uma equação para conseguirmos montar um sistema de equação adequado. Isso é obtido derivando a equação anterior em relação à ξ :

$$\begin{aligned} \varphi(\xi) = & + \frac{\partial u^*(\xi, x)}{\partial \xi} Q(x) \Big|_{x=L} - \frac{\partial u^*(\xi, x)}{\partial \xi} Q(x) \Big|_{x=0} \\ & - \frac{\partial \varphi^*(\xi, x)}{\partial \xi} M(x) \Big|_{x=L} + \frac{\partial \varphi^*(\xi, x)}{\partial \xi} M(x) \Big|_{x=0} \\ & + \frac{\partial M^*(\xi, x)}{\partial \xi} \varphi(x) \Big|_{x=L} - \frac{\partial M^*(\xi, x)}{\partial \xi} \varphi(x) \Big|_{x=0} \\ & + \int_0^L \frac{\partial u^*(\xi, x)}{\partial \xi} q(x) dx, \end{aligned}$$

onde:

$$\begin{aligned} \frac{\partial u^*(\xi, x)}{\partial \xi} &= \frac{r^2}{4EI} \left(\frac{\partial r}{\partial \xi} \right), \\ \frac{\partial \varphi^*(\xi, x)}{\partial \xi} &= \frac{r}{2EI} \left(\frac{\partial r}{\partial \xi} \right) \left(\frac{\partial r}{\partial x} \right), \\ \frac{\partial M^*(\xi, x)}{\partial \xi} &= \frac{1}{2} \left(\frac{\partial r}{\partial \xi} \right) \left(\frac{\partial r}{\partial x} \right)^2, \end{aligned}$$

onde $\frac{\partial r}{\partial \xi} = 1$ se $x < \xi$ e $\frac{\partial r}{\partial \xi} = -1$ se $x > \xi$.

```
function solfundviga(xi, x, E, I, L)
    r=abs(x-xi)
    if x==xi
        if xi==0
            drdx=1
        elseif xi==L
            drdx=-1
        else
            drdx=0
        end
    else
        drdx=(x-xi)/r
    end
    drdxi=-drdx

    u_star = r^3 / (12 * E * I)
    phi_star = (r^2 / (4 * E * I)) * drdx
    M_star = -r/2 * drdx^2
    Q_star = -1/2 * drdx^3

    du_star_dxi = (r^2 / (4 * E * I)) * drdxi
    dphi_star_dxi = (r / (2 * E * I)) * drdxi * drdx
    dM_star_dxi = 1/2 * drdxi * drdx^2

    u_star , phi_star , M_star , Q_star , du_star_dxi , dphi_star_dxi , dM_star_dxi
end
```

Considere uma viga com $L = 4$ m, $E = 50$ GPa, $I = 0.0036\text{m}^2$ e uma carga pontual no centro da viga de valor 10KN.

```
L=4
E=50e9
Iv=0.0036

#solfundviga(ξ, x,E,I,L)
u_00 ,phi_00 ,M_00 ,Q_00 ,du_00 ,dphi_00,dM_00= solfundviga(0,0,E,Iv,L)
u_0L ,phi_0L ,M_0L ,Q_0L ,du_0L ,dphi_0L,dM_0L= solfundviga(0,L,E,Iv,L)
u_L0 ,phi_L0 ,M_L0 ,Q_L0 ,du_L0 ,dphi_L0,dM_L0= solfundviga(L,0,E,Iv,L)
u_LL ,phi_LL ,M_LL ,Q_LL ,du_LL ,dphi_LL,dM_LL= solfundviga(L,L,E,Iv,L)
#multiplica U e phi
H=[-Q_00 -M_00 -Q_0L M_0L
    0 dM_00 0 -dM_0L
    Q_L0 -M_L0 -Q_LL M_LL
    0 dM_L0 0 -dM_LL]
#multiplica V e M
G=[-u_00 phi_00 u_0L -phi_0L
    -du_00 dphi_00 du_0L -dphi_0L
    -u_L0 phi_L0 u_LL -phi_LL
    -du_L0 dphi_L0 du_LL -dphi_LL]

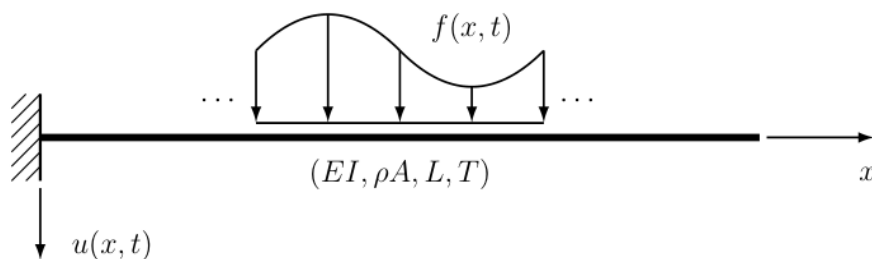
uc0 ,phic0 ,Mc0 ,Qc0 ,duc0 ,dphic0,dMc0=solfundviga(0,L/2,E,Iv,L)
ucL ,phicL ,McL ,QcL ,ducL ,dphicL,dMcL=solfundviga(L,L/2,E,Iv,L)
b=1e4*[uc0;duc0 ;ucL;ducL]

A=[-G[:,1] H[:,2] -G[:,3] H[:,4]]
x=A\b
```

5.1.1 Exercício

- 1- Generalize esse código para qualquer condição de contorno. Ele é capaz de resolver problemas hiperestáticos? Como?
- 2- Generalize esse código para qualquer carga distribuída e compare com a solução analítica.

6 Efeitos transientes



$$EI \frac{\partial^4 u}{\partial x^4} + \rho A \frac{\partial^2 u}{\partial t^2} = f(x, t)$$

Dividindo a equação por ρA e definindo

$$c = \sqrt{\frac{EI}{\rho A}}$$

obtem-se a expressão:

$$c^2 \frac{\partial^4 u}{\partial x^4} + \frac{\partial^2 u}{\partial t^2} = \frac{f(x, t)}{\rho A}$$

a solução fundamental para esse problema é dada por:

$$u^*(x, \xi, t, \tau) = \frac{1}{c} \left\{ \frac{r}{2} \left[S\left(\frac{r}{\sqrt{2\pi a}}\right) - C\left(\frac{r}{\sqrt{2\pi a}}\right) \right] + \frac{\sqrt{a}}{\sqrt{2\pi}} \left[\sin\left(\frac{r^2}{4a}\right) + \cos\left(\frac{r^2}{4a}\right) \right] \right\}$$

onde as funções S e C são denominadas integrais de Fresnel, $r = |x - \xi|$ e $a = c(t - \tau)$.

$$\begin{aligned} \theta^*(x, \xi, t, \tau) &= +\frac{1}{c} \left\{ \frac{1}{2} \left[S\left(\frac{r}{\sqrt{2\pi a}}\right) - C\left(\frac{r}{\sqrt{2\pi a}}\right) \right] \right\} \left(\frac{\partial r}{\partial x} \right), \\ M^*(x, \xi, t, \tau) &= -\frac{EI}{c} \left\{ \frac{1}{2\sqrt{2\pi a}} \left[\sin\left(\frac{r^2}{4a}\right) - \cos\left(\frac{r^2}{4a}\right) \right] \right\} \left(\frac{\partial r}{\partial x} \right)^2, \\ Q^*(x, \xi, t, \tau) &= -\frac{EI}{c} \left\{ \frac{r}{4\sqrt{2\pi a^3}} \left[\sin\left(\frac{r^2}{4a}\right) + \cos\left(\frac{r^2}{4a}\right) \right] \right\} \left(\frac{\partial r}{\partial x} \right)^3, \end{aligned}$$

A equação integral para esse problema é dada por:

$$\begin{aligned} u(\xi, t) &= -\frac{1}{\rho A} \int_0^t [u^* Q - \theta^* M + M^* \theta - Q^* u]_{x=0} d\tau \\ &\quad + \frac{1}{\rho A} \int_0^t [u^* Q - \theta^* M + M^* \theta - Q^* u]_{x=L} d\tau \\ &\quad + \frac{1}{\rho A} \int_0^t \int_0^L u^* f dx d\tau. \end{aligned}$$

A solução do problema, que envolve quatro incógnitas de contorno, requer ao menos duas equações integrais distintas. Assim, escreve-se, para a rotação:

$$\begin{aligned} \theta(\xi, t) &= -\frac{1}{\rho A} \left\{ \int_0^t \left[\frac{\partial u^*}{\partial \xi} Q - \frac{\partial \theta^*}{\partial \xi} M + \frac{\partial M^*}{\partial \xi} \theta - \frac{\partial Q^*}{\partial \xi} u \right]_{x=0} d\tau \right\} \\ &\quad + \frac{1}{\rho A} \left\{ \int_0^t \left[\frac{\partial u^*}{\partial \xi} Q - \frac{\partial \theta^*}{\partial \xi} M + \frac{\partial M^*}{\partial \xi} \theta - \frac{\partial Q^*}{\partial \xi} u \right]_{x=L} d\tau \right\} \\ &\quad + \frac{1}{\rho A} \left\{ \int_0^t \int_0^L \frac{\partial u^*}{\partial \xi} f dx d\tau \right\}, \end{aligned}$$

6.0.1 Exercício

3 - Faça um gráfico 3d de u^* e $du^*/d\xi$. Considere ξ e τ iguais a zero, que o tempo varia de 0 a 10s e que o x varia de 0 a L . Exemplo de superfície.

7 Desafio

1- Implemente um código que usando a formulação transiente e resolva um problema de vibração livre.

2-Esse problema também pode ser resolvido usando a solução fundamental estática e uma integral de domínio como feito para o problema de Poisson. Faça isso usando Houbolt e compare.

- Compare com uma solução analítica disponível no [Rao](#)

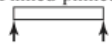
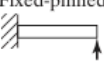
End Conditions of Beam	Frequency Equation	Mode Shape (Normal Function)	Value of $\beta_n l$
 Pinned-pinned	$\sin \beta_n l = 0$	$W_n(x) = C_n [\sin \beta_n x]$	$\beta_1 l = \pi$ $\beta_2 l = 2\pi$ $\beta_3 l = 3\pi$ $\beta_4 l = 4\pi$
 Free-free	$\cos \beta_n l \cdot \cosh \beta_n l = 1$	$W_n(x) = C_n [\sin \beta_n x + \sinh \beta_n x$ $+ \alpha_n (\cos \beta_n x + \cosh \beta_n x)]$ where $\alpha_n = \left(\frac{\sin \beta_n l - \sinh \beta_n l}{\cosh \beta_n l - \cos \beta_n l} \right)$	$\beta_1 l = 4.730041$ $\beta_2 l = 7.853205$ $\beta_3 l = 10.995608$ $\beta_4 l = 14.137165$ $(\beta l = 0 \text{ for rigid-body mode})$
 Fixed-fixed	$\cos \beta_n l \cdot \cosh \beta_n l = 1$	$W_n(x) = C_n [\sinh \beta_n x - \sin \beta_n x$ $+ \alpha_n (\cosh \beta_n x - \cos \beta_n x)]$ where $\alpha_n = \left(\frac{\sinh \beta_n l - \sin \beta_n l}{\cosh \beta_n l - \cos \beta_n l} \right)$	$\beta_1 l = 4.730041$ $\beta_2 l = 7.853205$ $\beta_3 l = 10.995608$ $\beta_4 l = 14.137165$
 Fixed-free	$\cos \beta_n l \cdot \cosh \beta_n l = -1$	$W_n(x) = C_n [\sin \beta_n x - \sinh \beta_n x$ $- \alpha_n (\cos \beta_n x - \cosh \beta_n x)]$ where $\alpha_n = \left(\frac{\sin \beta_n l + \sinh \beta_n l}{\cos \beta_n l + \cosh \beta_n l} \right)$	$\beta_1 l = 1.875104$ $\beta_2 l = 4.694091$ $\beta_3 l = 7.854757$ $\beta_4 l = 10.995541$
 Fixed-pinned	$\tan \beta_n l - \tanh \beta_n l = 0$	$W_n(x) = C_n [\sin \beta_n x - \sinh \beta_n x$ $+ \alpha_n (\cosh \beta_n x - \cos \beta_n x)]$ where $\alpha_n = \left(\frac{\sin \beta_n l - \sinh \beta_n l}{\cos \beta_n l - \cosh \beta_n l} \right)$	$\beta_1 l = 3.926602$ $\beta_2 l = 7.068583$ $\beta_3 l = 10.210176$ $\beta_4 l = 13.351768$
 Pinned-free	$\tan \beta_n l - \tanh \beta_n l = 0$	$W_n(x) = C_n [\sin \beta_n x + \alpha_n \sinh \beta_n x]$ where $\alpha_n = \left(\frac{\sin \beta_n l}{\sinh \beta_n l} \right)$	$\beta_1 l = 3.926602$ $\beta_2 l = 7.068583$ $\beta_3 l = 10.210176$ $\beta_4 l = 13.351768$ $(\beta l = 0 \text{ for rigid-body mode})$

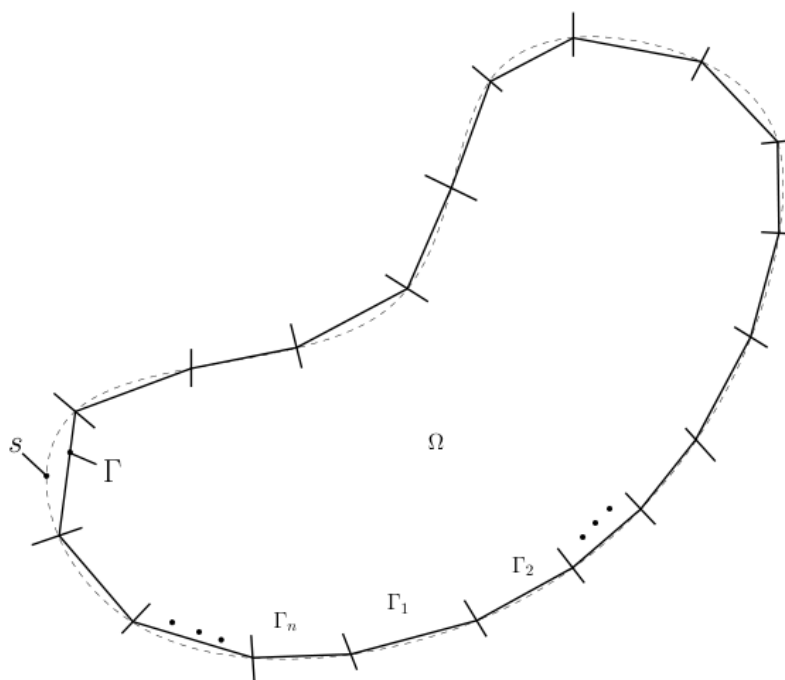
FIGURE 8.15 Common boundary conditions for the transverse vibration of a beam.

3-Implemente a formulação da [viga de Timoshenko](#) e de [BICKFORD-REDDY](#) e compare as três para diferentes tamanhos de viga.

8 Indo para 2D

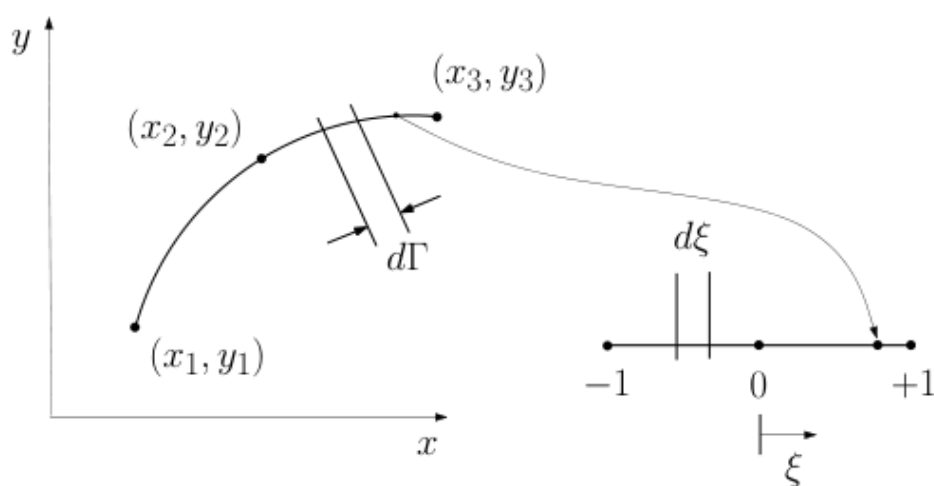
8.1 Cálculo do perímetro de figuras planas

Uma vez que a integral ao longo do contorno no s pode ser bastante difícil, ou mesmo impossível, uma estratégia para o cálculo do perímetro é a divisão do contorno em uma soma de pequenos pedaços $s_1, s_2, \dots, s_n$, ou seja, $s = \sum_{i=1}^n s_i$, onde n é o número de pedaços em que o contorno foi dividido. Uma vez que estes pedaços podem ter uma forma qualquer, cada pedaço s_i será aproximado por uma forma conhecida. Por simplicidade, esta forma é quase sempre dada por um polinômio (linha reta, parábola, etc).



Dessa maneira, cada pedaço $s_1, s_2, \dots, s_n$ é aproximado por formas conhecidas $\Gamma_1, \Gamma_2, \dots, \Gamma_n$, chamados Elementos de Contorno.

Considere agora que os elementos de contorno Γ_i sejam parabólicos, ou seja, são descritos por polinômios de 2ª ordem (equação de uma parábola). Desta forma, são necessários 3 pontos de Γ_i para se definir a parábola. Estes pontos são dados por (x_1, y_1) , (x_2, y_2) e (x_3, y_3) , que correspondem respectivamente a $\xi = -1$, $\xi = 0$ e $\xi = 1$.



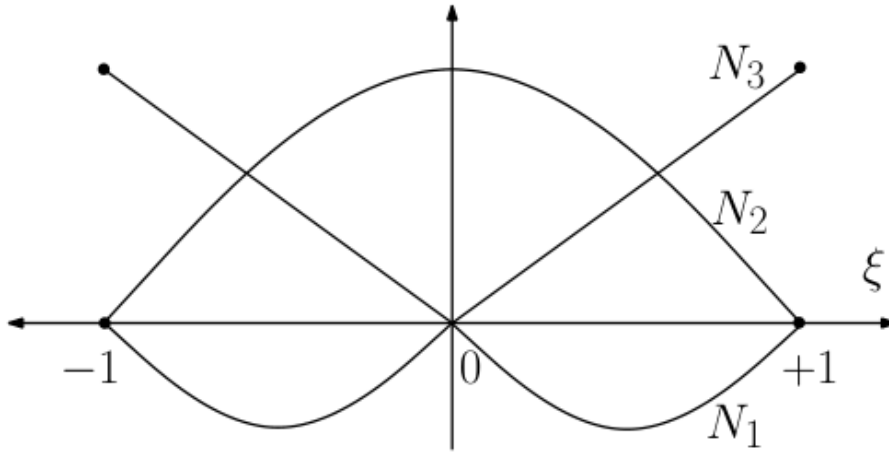
Criando uma função parabólica para relacionar x com ξ , tem-se: $x = a\xi^2 + b\xi + c$ sendo que:

$$x(\xi = -1) = x_1 = a(-1)^2 + b(-1) + c$$

$$x(\xi = 0) = x_2 = a(0)^2 + b(0) + c$$

$$x(\xi = +1) = x_3 = a(+1)^2 + b(+1) + c$$

resolvendo esse sistema obtém-se $x = N_1x_1 + N_2x_2 + N_3x_3$ onde $N_1 = \frac{\xi}{2}(\xi - 1)$, $N_2 = (1 - \xi)(1 + \xi)$ e $N_3 = \frac{\xi}{2}(\xi + 1)$ são as funções de forma quadráticas contínuas.



A derivada de x em relação a ξ é dada por $\frac{dx}{d\xi} = \frac{d[N_1(\xi)]}{d\xi}x_1 + \frac{d[N_2(\xi)]}{d\xi}x_2 + \frac{d[N_3(\xi)]}{d\xi}x_3$ onde $\frac{dN_1}{d\xi} = \xi - \frac{1}{2}$, $\frac{dN_2}{d\xi} = -2\xi$ e $\frac{dN_3}{d\xi} = \xi + \frac{1}{2}$.

O comprimento Γ do perímetro da figura é então dado por:

$\Gamma = \sum_{i=1}^n \int_{-1}^1 \frac{d\Gamma}{d\xi} d\xi = \sum_{i=1}^n \int_{-1}^1 \sqrt{\left(\frac{dx}{d\xi}\right)^2 + \left(\frac{dy}{d\xi}\right)^2} d\xi = \sum_{i=1}^n \left[\int_{-1}^1 J(\xi) d\xi \right]$ em que a função $J(\xi) = \sqrt{\left(\frac{dx}{d\xi}\right)^2 + \left(\frac{dy}{d\xi}\right)^2}$ é chamada jacobiano da transformação. Usando quadratura de Gauss com p pontos de Gauss, conclui-se que $\Gamma = \sum_{i=1}^n \left[\sum_{k=1}^p w_k J(\xi_k) \right]$.

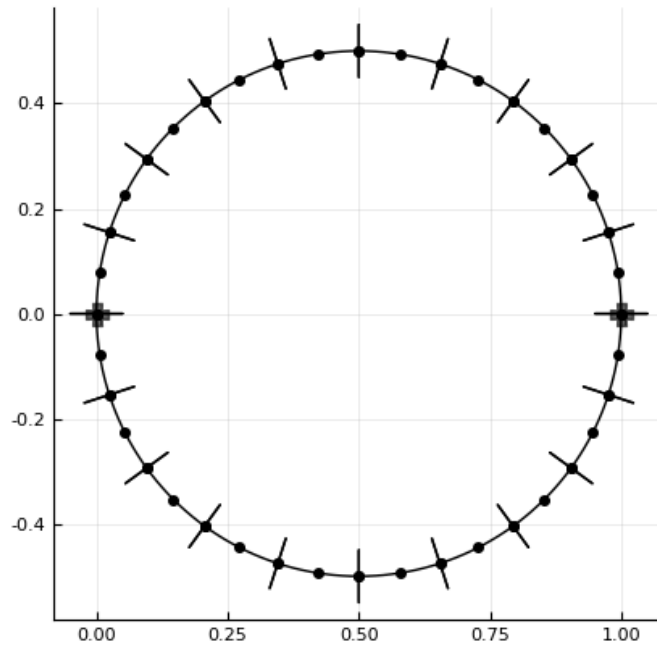
código para propriedade geométrica

```
include("dad.jl")
include("format.jl")
include("calcula_PropGeom.jl")
include("calc_fforma.jl")
using FastGaussQuadrature, GLMakie

PONTOS, SEGMENTOS, MALHA = dad_1(10, 1) #Arquivo de entrada de dados

NOS, ELEM = format_dad(PONTOS, SEGMENTOS, MALHA) # formata os dados (cria as
# matrizes NOS e ELEM a partir das matrizes PONTOS, SEGMENTOS e MALHA)
display(mostra_geo(SEGMENTOS, PONTOS, ELEM, NOS))

Perimetro, Area, xbarra, ybarra = calcula_PropGeom(ELEM, NOS, 4, 4); # Calcula as
propriedades geométricas de figuras plana
println("Perimetro calculado numericamente: $Perimetro")
println("Área calculada numericamente: $Area")
println("Centróide calculado numericamente: ( $xbarra, $ybarra)")
```



8.2 Área

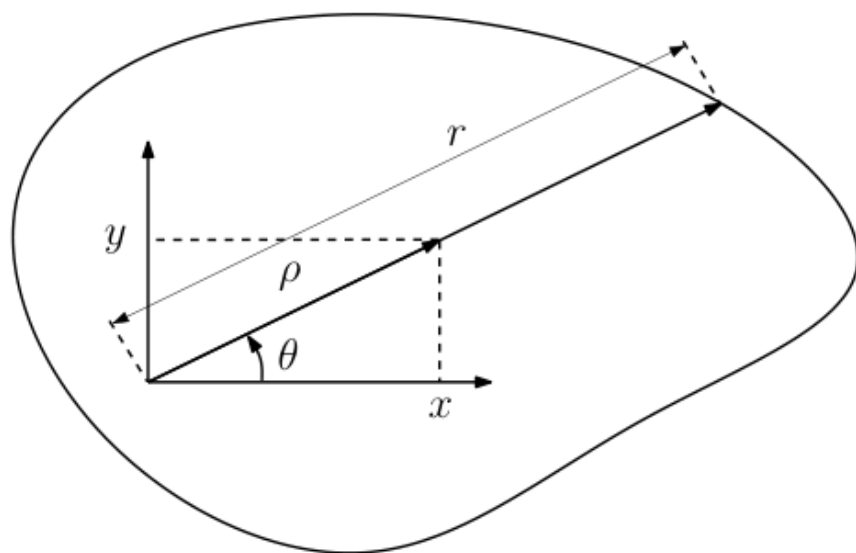
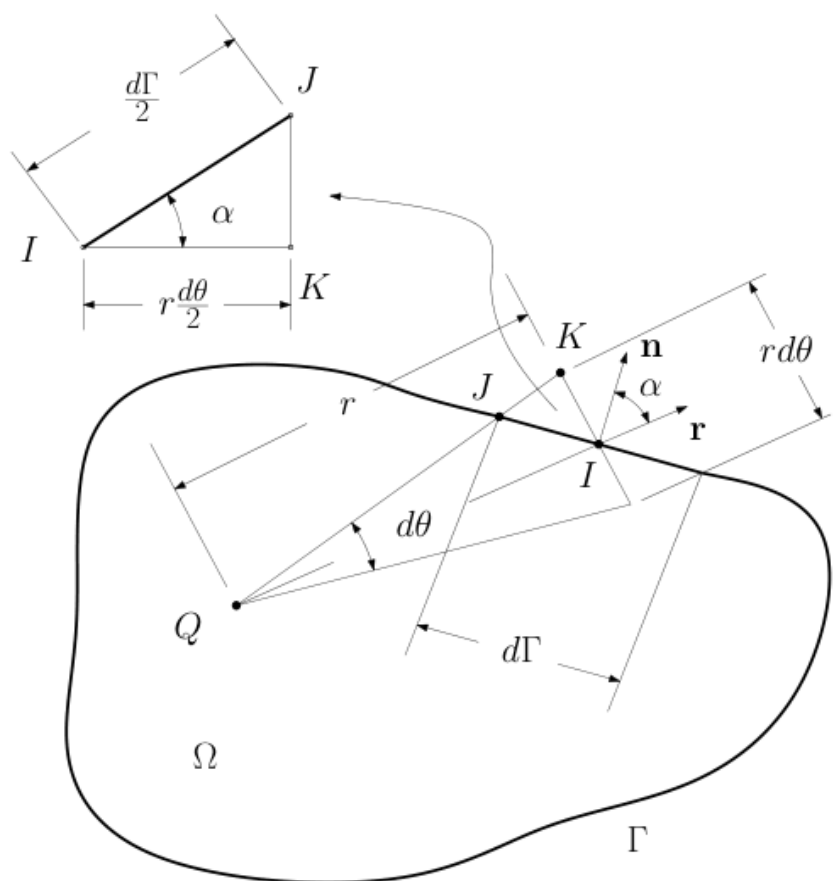
A integral de uma função $f(x, y)$ sobre a área Ω de uma figura plana é dada pela integral $I = \int_{\Omega} f(x, y) dx dy$. A integral pode ser escrita, em coordenadas polares, como: $I = \int_{\Omega} f(x(\rho, \theta), y(\rho, \theta)) \rho d\rho d\theta = \int_{\theta} \int_0^r f(x(\rho, \theta), y(\rho, \theta)) \rho d\rho d\theta$ definindo F como $F(\rho, \theta) = \int_0^r f(x(\rho, \theta), y(\rho, \theta)) \rho d\rho$, pode-se escrever: $I = \int_{\theta} F(\rho, \theta) d\theta$.

como $\cos \alpha = \frac{r \frac{d\theta}{2}}{\frac{d\Gamma}{2}}$

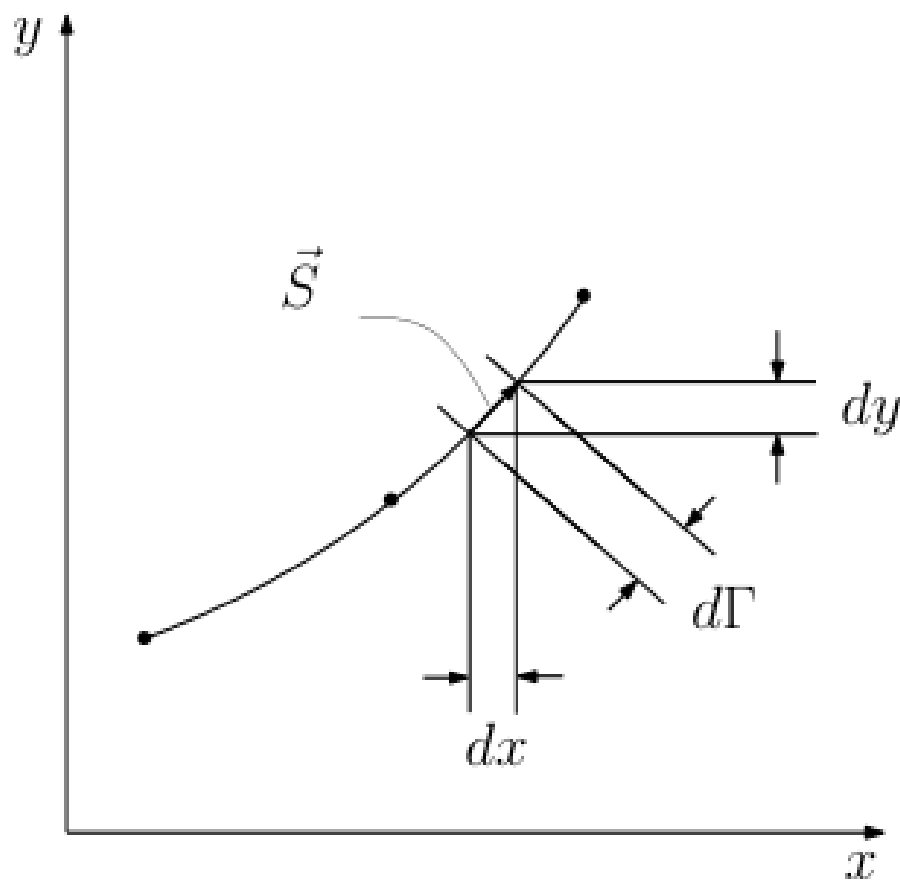
podemos escrever: $d\theta = \frac{\vec{n} \cdot \vec{r}}{r} d\Gamma$ e finalmente $I = \int_{\Gamma} F \frac{\vec{n} \cdot \vec{r}}{r} d\Gamma$.

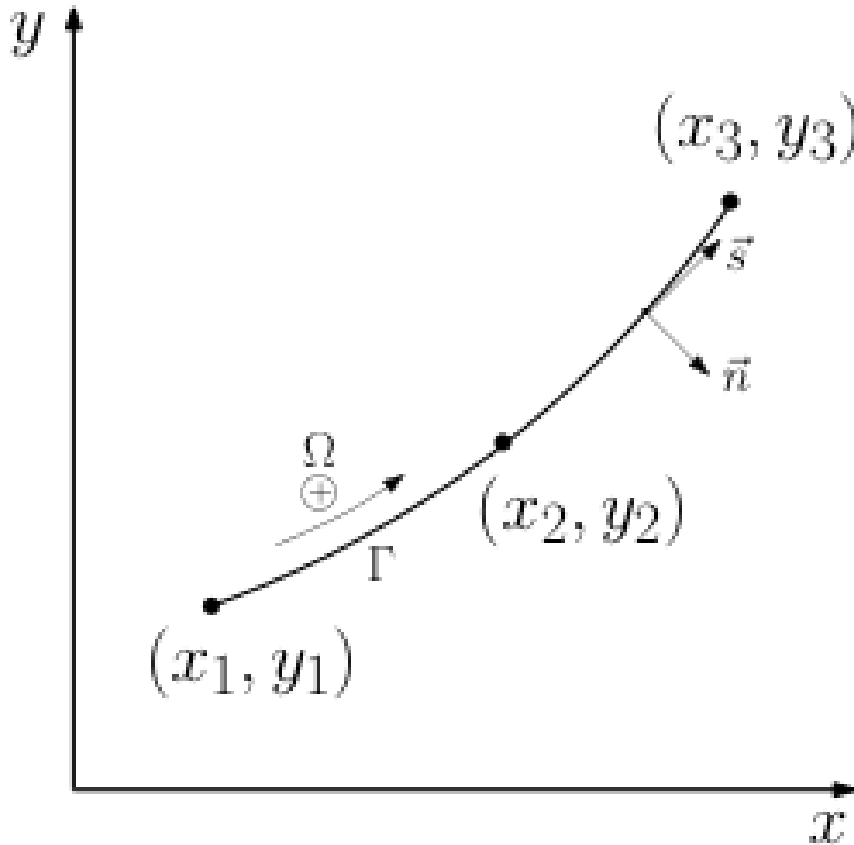
Dividindo Γ em uma soma de pequenos pedaços do contorno:

$$I = \sum_{i=1}^{NE} \int_{\Gamma_i} F \frac{\vec{n} \cdot \vec{r}}{r} d\Gamma.$$



8.2.1 Cálculo do vetor normal $\vec{n}$





$\vec{S}$ = vetor tangente ao contorno Γ . $\vec{S} = dx\vec{i} + dy\vec{j}$ vetor não unitário

$\vec{s}$ = Vetor unitário tangente ao contorno Γ

$$\vec{s} = \frac{\vec{S}}{|\vec{S}|} = \frac{dx\vec{i} + dy\vec{j}}{\sqrt{dx^2 + dy^2}}$$

Dividindo tudo por $d\xi$, tem-se:

$$\vec{s} = \frac{\frac{dx}{d\xi}}{\sqrt{\left(\frac{dx}{d\xi}\right)^2 + \left(\frac{dy}{d\xi}\right)^2}}\vec{i} + \frac{\frac{dy}{d\xi}}{\sqrt{\left(\frac{dx}{d\xi}\right)^2 + \left(\frac{dy}{d\xi}\right)^2}}\vec{j}$$

$$\vec{s} = \frac{\frac{dx}{d\xi}}{\frac{d\Gamma}{d\xi}}\vec{i} + \frac{\frac{dy}{d\xi}}{\frac{d\Gamma}{d\xi}}\vec{j} = s_x\vec{i} + s_y\vec{j}$$

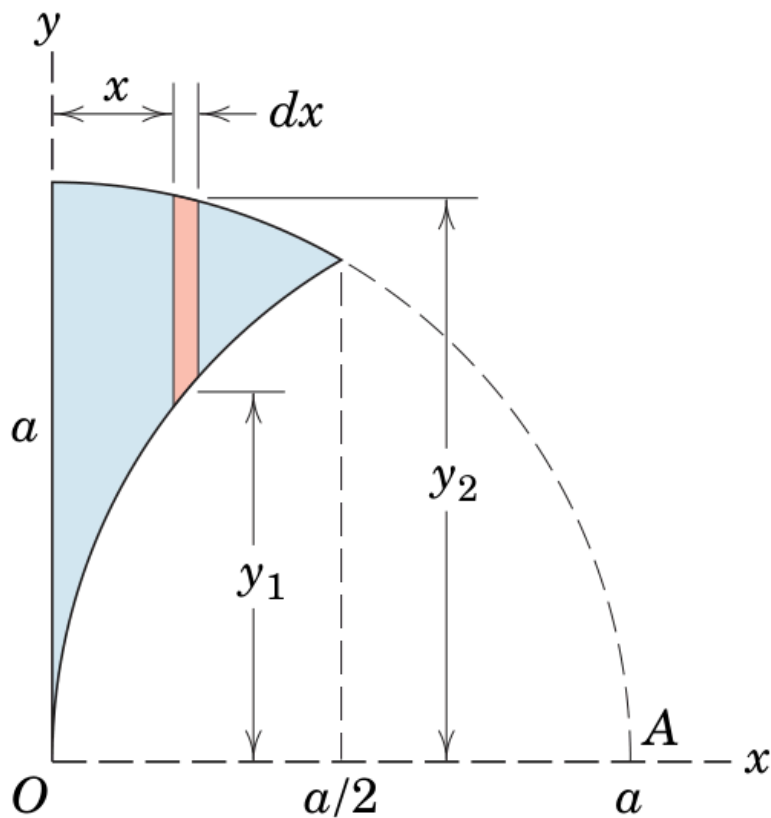
$\vec{n}$ = vetor unitário normal ao contorno Γ apontando para fora do domínio Ω

$\vec{n} = n_x\vec{i} + n_y\vec{j}$ Temos duas possibilidades, uma vez que $\vec{s}$ é unitário:

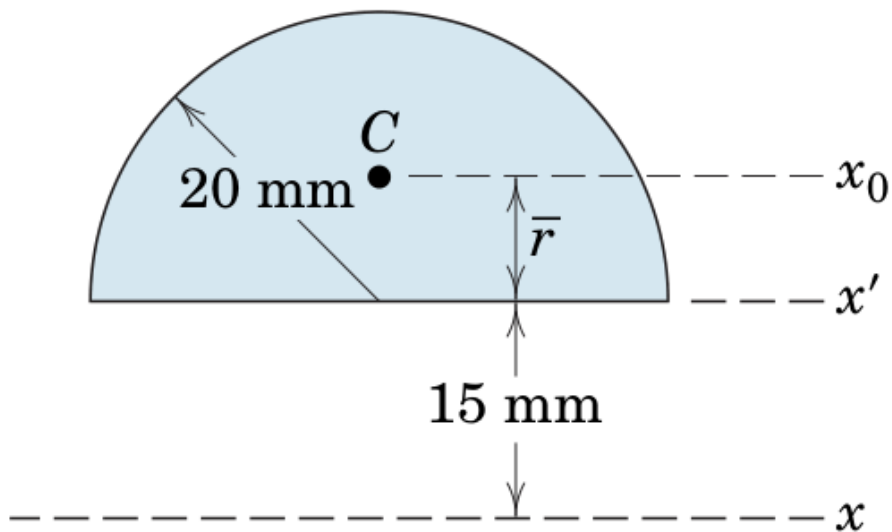
$$n_x = -s_y \quad n_y = s_x \Rightarrow (-s_y)s_x + (s_x)s_y = 0$$

$n_x = s_y \quad n_y = -s_x \Rightarrow (s_y)s_x + (-s_x)s_y = 0$ O vetor $\vec{s}$ é sempre definido percorrendo Γ no sentido anti-horário, se Γ for um contorno externo e horário, caso Γ seja um contorno interno. Portanto, a segunda possibilidade é escolhida de modo que a normal esteja apontada para fora do domínio.

8.2.2 Exercícios



$$I_x = \frac{a^4}{96} (9\sqrt{3} - 2\pi)$$



$$I_x = 2 \cdot 10^4 \pi - \frac{20^2 \cdot \pi}{2} \left(\frac{80}{3 \cdot \pi} \right)^2 + \left(\frac{20^2 \pi}{2} \right) \left(15 + \frac{80}{3 \pi} \right)^2$$

Calcule, modificando o programa propgeio, os momentos de inércia em relação ao eixo x para as duas figuras e compare com o resultado analítico.

- Gravações <https://youtu.be/TZvHS2JzKOs> <https://youtu.be/08vADm8JguE> <https://youtu.be/O14y7y9ZCLs> <https://youtu.be/YbKmFAwBbw0>

9 Laplace 2D

- Código <https://github.com/l-s-campos/BEM> faça o download e rode de dentro da pasta:

```
using Pkg

Pkg.activate(pwd())

Pkg.instantiate()
```

@which

10 Formulação

A equação de Laplace é dada por:

$$\nabla^2 T = 0,$$

usando o método dos resíduos ponderados e aplicando a segunda identidade de Green:

$$\int_{\Omega} (v \nabla^2 u - u \nabla^2 v) d\Omega = \int_{\Gamma} \left(v \frac{\partial u}{\partial n} - u \frac{\partial v}{\partial n} \right) ds$$

obtem-se a equação integral de contorno:

$$0.5T(x_d, y_d) = \int_{\Gamma} T q^* ds - \int_{\Gamma} T^* q ds,$$

onde T^* e q^* são as soluções fundamentais:

$$T^* = \frac{-1}{2\pi k} \ln r$$

$$q^* = \frac{1}{2\pi r^2} [(x - x_d)n_x + (y - y_d)n_y].$$

Podemos representar a temperatura e fluxo em um elemento descontínuo com m nós como: $T = N_1 T_1 + N_2 T_2 + N_3 T_3 + \dots + N_m T_m$ $q = N_1 q_1 + N_2 q_2 + N_3 q_3 + \dots + N_m q_m$ onde estamos aproximando nossa distribuição de temperatura e fluxo no elemento por uma função polinomial de grau $(m - 1)$.

Discretizando em n elementos de contorno descontínuos:

$$0.5T(x_d, y_d) = \sum_{j=1}^{n_{elem}} \left[\int_{\Gamma_j} T q^* d\Gamma \right] - \sum_{j=1}^{n_{elem}} \left[\int_{\Gamma_j} T^* q d\Gamma \right]$$

Usando a representação de T e q na equação acima temos:

$$0.5T(x_d, y_d) = \sum_{j=1}^{n_{elem}} \left\{ \int_{\Gamma_j} [N_1 \ N_2 \ N_3 \ \dots \ N_m] \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_m \end{bmatrix}_j q^* d\Gamma \right\} - \sum_{j=1}^{n_{elem}} \left\{ \int_{\Gamma_j} T^* [N_1 \ N_2 \ N_3 \ \dots \ N_m] \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ \vdots \\ q_m \end{bmatrix}_j d\Gamma \right\}$$

Repetindo essa equação para cada diferente nó podemos montar um sistema matricial:

$$HT = Gq$$

10.1 Método indireto para o cálculo da diagonal da matriz H

A diagonal da matriz H contém uma singularidade forte que dificulta o cálculo numérico desses termos. Uma alternativa não faz a integração de maneira explícita mas usa uma propriedade da matriz H decorrente da modelagem de um corpo sob temperatura constante. Sem perder a generalidade, considere que todos os nós de um corpo encontrem-se com a temperatura $T = 1$. Neste caso, o fluxo será nulo em todos os nós, ou seja, $q = 0$ em todos os nós. Desta forma, a equação matricial é reescrita como $H\{1\} = G\{0\}$.

Daí, os termos da diagonal da matriz $[H]$ pode ser calculado da seguinte forma:

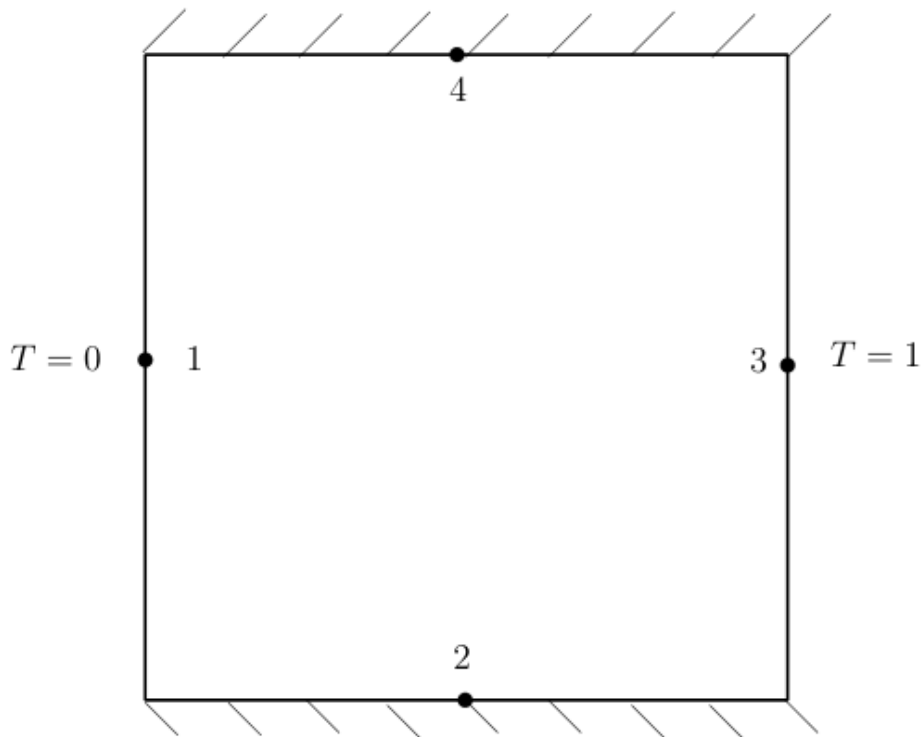
$$H_{ii} = - \sum_{j=1}^N H_{ij}, \text{ com } i \neq j, \text{ para } i = 1, 2, \dots, N,$$

uma vez que todos os termos de fora da diagonal são integrais regulares e já foram previamente calculados.

Um procedimento parecido, usando uma outra distribuição de temperatura, pode ser feito com a diagonal da matriz G mas isso normalmente não é necessário por se tratar de uma singularidade fraca

10.2 Exemplo

A fim de ilustrar como se aplica as condições de contorno e se calcula as variáveis desconhecidas será analisado um problema de condução de calor unidirecional com uma discretização de um elemento por lado.



As equações obtidas podem ser escritas na forma matricial, como:

$$\begin{pmatrix} H_{11} & H_{12} & H_{13} & H_{14} \\ H_{21} & H_{22} & H_{23} & H_{24} \\ H_{31} & H_{32} & H_{33} & H_{34} \\ H_{41} & H_{42} & H_{43} & H_{44} \end{pmatrix} \begin{pmatrix} \bar{T}_1 \\ \bar{T}_2 \\ \bar{T}_3 \\ \bar{T}_4 \end{pmatrix} = \begin{pmatrix} G_{11} & G_{12} & G_{13} & G_{14} \\ G_{21} & G_{22} & G_{23} & G_{24} \\ G_{31} & G_{32} & G_{33} & G_{34} \\ G_{41} & G_{42} & G_{43} & G_{44} \end{pmatrix} \begin{pmatrix} q_1 \\ \bar{q}_2 \\ q_3 \\ \bar{q}_4 \end{pmatrix}$$

onde $\bar{T}$ e $\bar{q}$ são termos conhecidos.

Separando os termos conhecidos dos desconhecidos:

$$\begin{pmatrix} -G_{11} & H_{12} & -G_{13} & H_{14} \\ -G_{21} & H_{22} & -G_{23} & H_{24} \\ -G_{31} & H_{32} & -G_{33} & H_{34} \\ -G_{41} & H_{42} & -G_{43} & H_{44} \end{pmatrix} \begin{pmatrix} q_1 \\ T_2 \\ q_3 \\ T_4 \end{pmatrix} = \begin{pmatrix} -H_{11} & G_{12} & -H_{13} & G_{14} \\ -H_{21} & G_{22} & -H_{23} & G_{24} \\ -H_{31} & G_{32} & -H_{33} & G_{34} \\ -H_{41} & G_{42} & -H_{43} & G_{44} \end{pmatrix} \begin{pmatrix} \bar{T}_1 \\ \bar{q}_2 \\ \bar{T}_3 \\ \bar{q}_4 \end{pmatrix}$$

Assim, pode-se escrever $Ax = b$ e resolvendo o sistema linear calcula-se os valores das variáveis desconhecidas.

10.3 Pontos internos

A equação integral para pontos internos é ligeiramente modificada:

$$T(x_d, y_d) = \int_{\Gamma} T q^* ds - \int_{\Gamma} T^* q ds$$

Aqui temos um detalhe importante, uma vez conhecida T e q no contorno a temperatura pode ser calculada em qualquer ponto interno sem ser necessário resolver algum sistema linear.

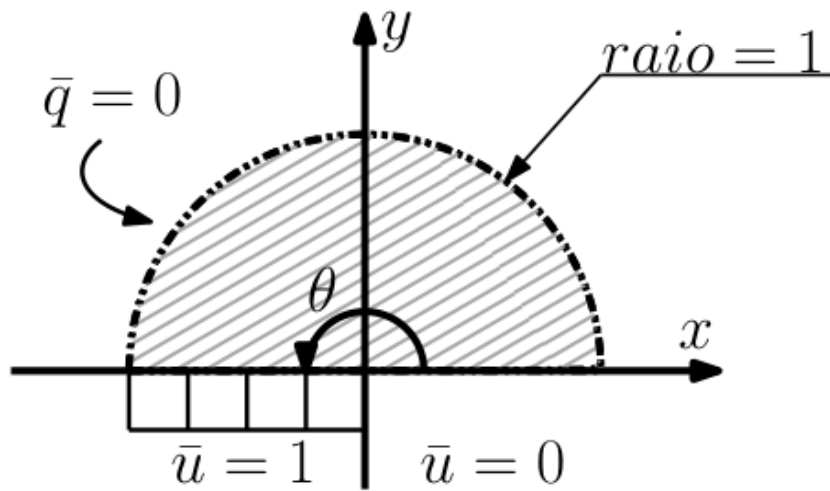
10.4 Exercícios

erros

1. Resolva esse problema com elementos lineares, quadráticos e cúbicos e calcule o erro médio, erro máximo e a norma l2 do erro no contorno para diferentes discretizações. Faça um gráfico comparando a convergência dos dos 3 tipos de elementos. A solução analítica é dada por:

$$u(\theta) = \frac{\theta}{\pi}$$

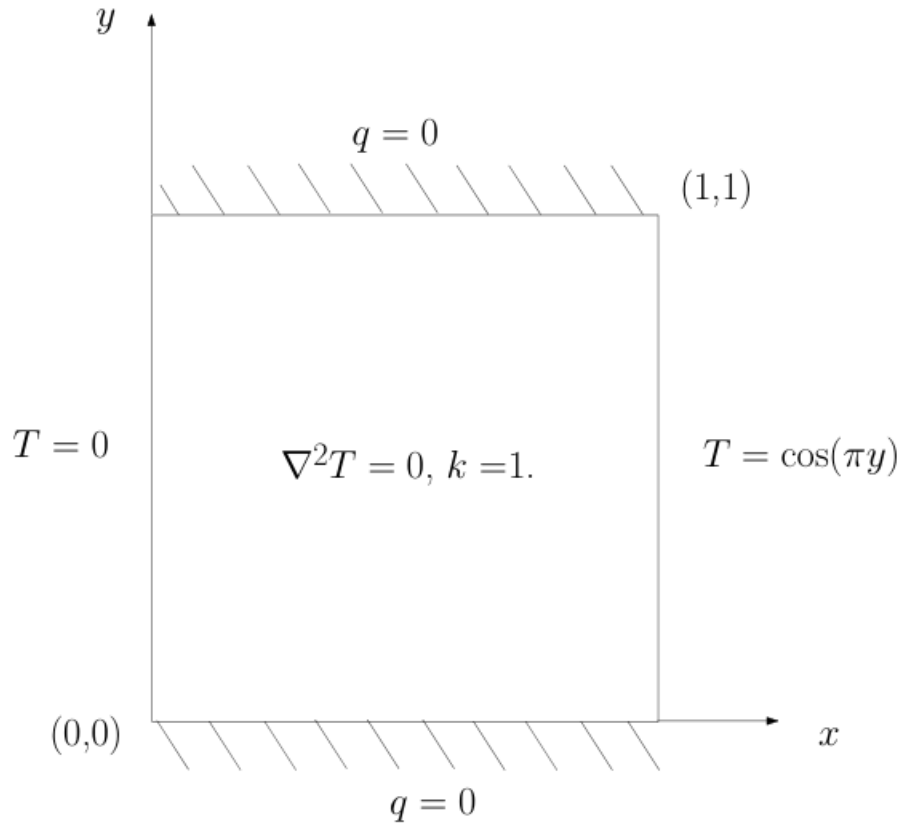
$$q(x) = -\frac{1}{\pi x} \quad \text{em } y = 0$$



1. Analise o seguinte problema:

Analise uma placa com as seguintes condições de contorno:

- $T(x = 0) = 0$
- $T(x = 1) = \cos(\pi y)$
- $q(y = 0) = q(y = 1) = -k \frac{\partial u}{\partial n} = 0$

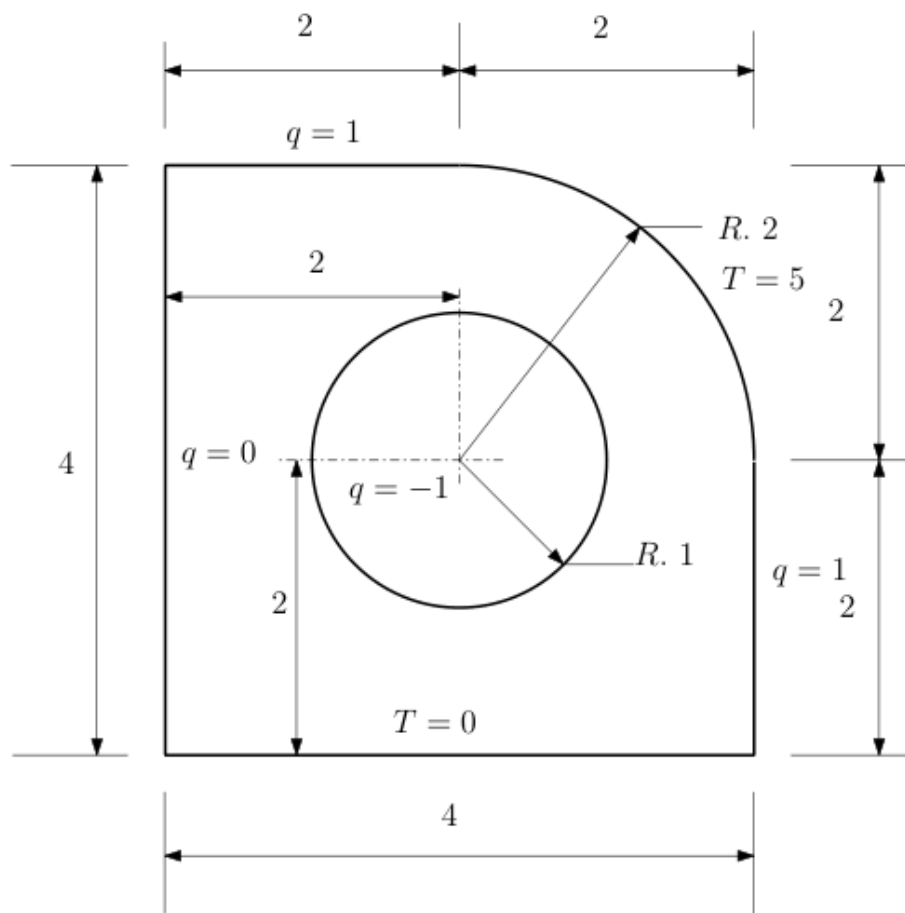


A solução analítica para este problema é dada por:

$$T^{an} = \sinh(\pi x) \cos(\pi y) / \sinh(\pi)$$

Analise o problema usando diferentes números de elementos e faça uma tabela mostrando o erro percentual em um ponto interno de coordenadas $(x, y) = (\sqrt{2}/2, \sqrt{2}/2)$ para os diversos casos analisados.

1. Faça um mapa de cor da distribuição de temperatura em uma placa com dimensões e condições de contorno mostradas na figura.



10.5 Desafio

Baseado nesse [artigo](#) calcule o coeficiente de sustentação de um perfil NACA usando o BEM.

[gravação](#)

11 Medidas de erro

erro médio

$$\| \mathbf{u} \|_{med} = \sum | u_i |$$

$$\varepsilon_u = \frac{\| \mathbf{u} - \mathbf{u}^{exact} \|_{med}}{\| \mathbf{u}^{exact} \|_{med}}$$

erro máximo

$$\| \mathbf{u} \|_{max} = \max | u_i |$$

$$\varepsilon_u = \frac{\| \mathbf{u} - \mathbf{u}^{exact} \|_{max}}{\| \mathbf{u}^{exact} \|_{max}}$$

norma l2 do erro

$$\| \mathbf{u} \|_{L_2} = \sqrt{\sum_{i=1}^{N_e} (u_i)^2}$$

$$\varepsilon_u = \frac{\| \mathbf{u} - \mathbf{u}^{exact} \|_{L_2}}{\| \mathbf{u}^{exact} \|_{L_2}}$$

12 Poisson 2D

13 MECID

A equação de Poisson é dada por:

$$\nabla^2 T = f(x, y),$$

um novo termo aparece na equação integral de contorno:

$$0.5T(x_d, y_d) = \int_{\Gamma} T q^* ds - \int_{\Gamma} T^* q ds + \int_{\Omega} T^* f d\Omega,$$

esse termo precisa de um tratamento extra. Vamos aproximar $T^* f$ por funções de base radial:

$$T^* f = \sum_{j=1}^n \phi(r_j) \alpha_j.$$

ϕ é uma função de base radial que será tomada aqui por $r^2 \log(r)$. α são coeficientes desconhecidos que podem ser obtidos aplicando essa equação n vezes e montando um sistema matricial:

$$\begin{bmatrix} X_1 - X_1 & \phi(X_1 - X_2) & \cdots & \phi(X_1 - X_N) \\ X_2 - X_1 & \phi(X_2 - X_2) & \cdots & \phi(X_2 - X_N) \\ \vdots & \vdots & \ddots & \vdots \\ X_N - X_1 & \phi(X_N - X_2) & \cdots & \phi(X_N - X_N) \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_N \end{bmatrix} = \begin{bmatrix} f(X_1)T^*(X_d, X_1) \\ f(X_2)T^*(X_d, X_2) \\ \vdots \\ f(X_N)T^*(X_d, X_N) \end{bmatrix} = \begin{bmatrix} T^*(X_d, X_1) & \cdots & 0 \\ 0 & T^*(X_d, X_2) & 0 \\ \vdots & \ddots & \vdots \\ 0 & 0 & T^*(X_d, X_N) \end{bmatrix} \begin{bmatrix} f(X_1) \\ f(X_2) \\ \vdots \\ f(X_N) \end{bmatrix}$$

escrito de maneira sintética:

$$\alpha = F^{-1} Df.$$

Voltando a integral de domínio:

$$\int_{\Omega} T^* f d\Omega = \int_{\Omega} \sum_{j=1}^n \phi(r_j) \alpha_j d\Omega = \sum_{j=1}^n \int_{\Omega} \phi(r_j) d\Omega F^{-1} Df = s F^{-1} Df = s_f Df$$

$$= \{s_1 \ s_2 \ \dots \ s_n\} F^{-1} \begin{bmatrix} T^*(X_d, X_1) & \cdots & 0 \\ 0 & T^*(X_d, X_2) & 0 \\ \vdots & \ddots & \vdots \\ 0 & 0 & T^*(X_d, X_N) \end{bmatrix} \begin{bmatrix} f(X_1) \\ f(X_2) \\ \vdots \\ f(X_N) \end{bmatrix} = \{s_{f1} T^*(X_d, X_1) \ s_{f2} T^*(X_d, X_2) \ \dots \ s_{fN} T^*(X_d, X_N)\} \begin{bmatrix} f(X_1) \\ f(X_2) \\ \vdots \\ f(X_N) \end{bmatrix}$$

Fazendo isso para os n pontos fontes:

$$\begin{bmatrix} s_{f1} T^*(X_1, X_1) & s_{f2} T^*(X_1, X_2) & \cdots & s_{fN} T^*(X_1, X_N) \\ s_{f1} T^*(X_2, X_1) & s_{f2} T^*(X_2, X_2) & \cdots & s_{fN} T^*(X_2, X_N) \\ \vdots & \vdots & \ddots & \vdots \\ s_{f1} T^*(X_N, X_1) & s_{f2} T^*(X_N, X_2) & \cdots & s_{fN} T^*(X_N, X_N) \end{bmatrix} \begin{bmatrix} f(X_1) \\ f(X_2) \\ \vdots \\ f(X_N) \end{bmatrix} = M f$$

a diagonal dessa matriz não é bem definida devido a singularidade da solução fundamental. Ela será calculada de maneira indireta como feito com a matriz H .

Esse procedimento tem de ser capaz de calcular essa integral quando a função f for constante e igual a 1.

$$I_{1d} = \int_{\Omega} T^*(X_d, X) d\Omega$$

Igualando as equações:

$$M[1] = [I_1]$$

a diagonal da matriz M tem de ser dada por

$$M_{ii} = I_{1i} - \sum_{j=1}^N M_{ij}, \text{ com } i \neq j, \text{ para } i = 1, 2, \dots, N,$$

```
Ht, Gt = BEM.calc_HeGt(dad)
A, b = BEM.aplicaCDC(Ht, Gt, dad)

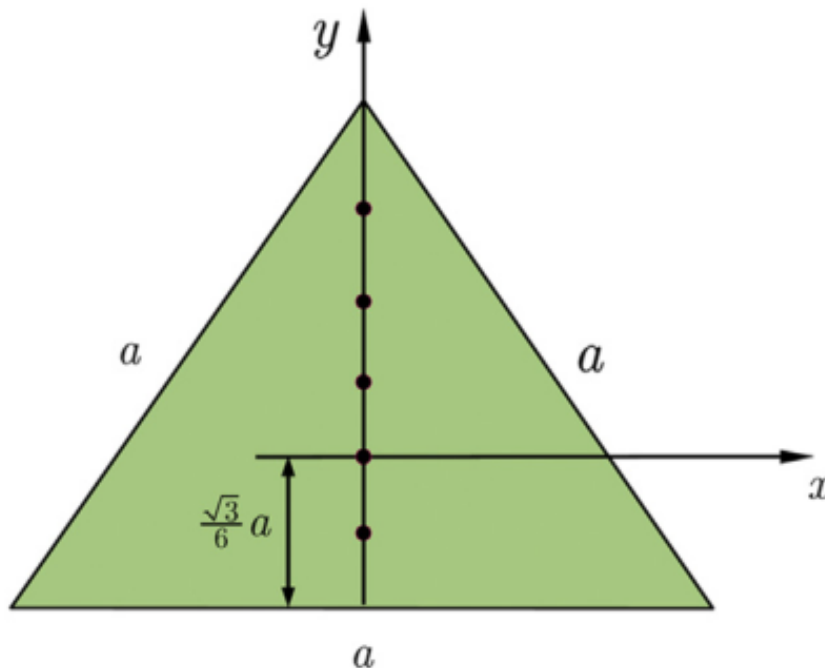
M = BEM.Monta_M_RIMd(dad, npg) # calc_HeG_potencial linha 310
f=ones(nc(dad)+ni(dad))*10
x = A \ (b+M*f) #carga distribuída
x = A \ (b+M*u) #difusão
T, q = separa(dad, x) #format 479
Ti=x[nc(dad)+1:end]
```

13.1 Exercícios

Todos os exercícios tem de ser resolvidos com diferentes discretizações.

1. Determine a superfície de deflexão de uma membrana elástica em forma de um triângulo equilátero com comprimento lateral $a = 5,0$ m. A membrana está fixa ao longo de sua borda e é submetida a uma carga distribuída uniformemente $f = 10$ kN/m² e uma tensão $S = 1$ kN/m. Os eixos coordenados são tomados como mostrado na figura.

A equação que rege esse problema é: $S \nabla^2 w = -f$

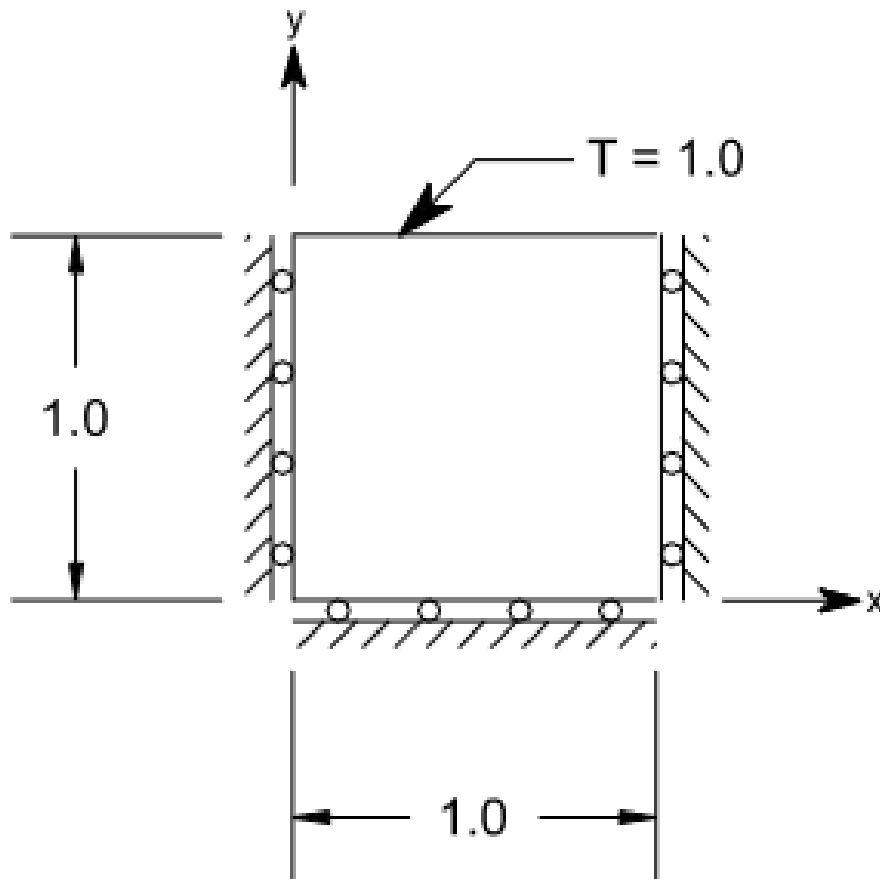


A solução analítica é dada por:

$$w = -\frac{f}{2S} \left[\frac{1}{2}(x^2 + y^2) - \frac{1}{a\sqrt{3}}(y^3 - 3x^2y) - \frac{1}{18}a^2 \right]$$

Calcule o erro médio e a norma L2 do erro em 100 pontos internos. Faça um gráfico com a distribuição de deflexão.

1. Considere um cubo unitário inicialmente a temperatura zero e submetido a um aquecimento súbito em uma de suas faces.



A face aquecida é elevada e mantida a uma temperatura unitária. Supõe-se que as propriedades do material sejam unitárias. A solução analítica para este problema de exemplo pode ser encontrada como:

$$T(Y, t) = 1 - \frac{4}{\pi} \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} \exp \left\{ -\frac{(2n+1)^2 \pi^2 \kappa t}{4L^2} \right\} \cos \frac{(2n+1)\pi Y}{2L}$$

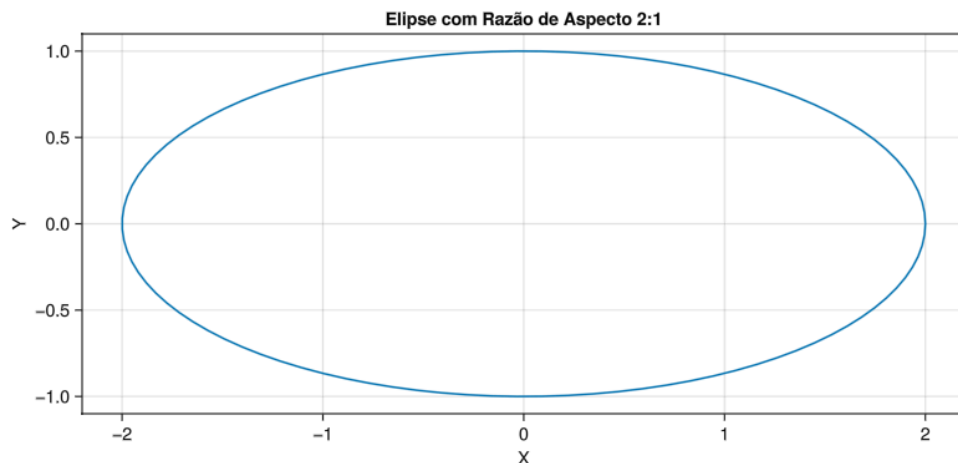
Usando algum método de análise transiente da aula 2, resolva esse problema e compare com a solução analítica.

1. Considere uma elipse que é governada por: $\nabla^2 u = 4 - x^2$

a solução analítica é dada por:

$$u = \left[1.6 - \frac{1}{246}(50x^2 - 8y^2 + 33.6) \right] \left(\frac{x^2}{4} + y^2 - 1 \right) e^{q = 0.4(x^2 + 8y^2) + \frac{1}{246}(-50x^3 - 96xy^2 + 83.2x)\frac{x}{2} + \frac{1}{246}(-96x^2y + 32y^3 - 83.2y)y}$$

calcule os erros do potencial nos pontos internos e do fluxo no contorno.



extra

Um cilindro oco com temperatura inicial zero é considerado. O raio interno $a=1$ e o raio externo $b = 2$. A superfície interna do cilindro é mantida a uma temperatura=1. Nesse caso, são impostas restrições de simetria. As propriedades do material são unitárias. A solução analítica é dada por:

$$T(r, t) = \frac{\ln(b/r)}{\ln(b/a)} T_1 + \pi \sum_{n=1}^{\infty} \frac{J_0(b\alpha_n) J_0(a\alpha_n)}{J_0^2(a\alpha_n) - J_0^2(b\alpha_n)} \{ J_0(r\alpha_n) Y_0(b\alpha_n) - J_0(b\alpha_n) Y_0(r\alpha_n) \} e^{-i\alpha_n^2 t},$$

onde T_i é a temperatura constante na superfície interna, J_0 e Y_0 são as funções de Bessel de primeira e segunda espécies, respectivamente, e α é a raiz de:

$$J_0(ax)Y_0(bx) - J_0(bx)Y_0(ax) = 0.$$

you pode usar a função `fzeros` para encontrar α

13.2 Gravação

https://youtu.be/uSREar_ejnM

14 Elasticidade 2D

15 Equações importantes

A relação de equilíbrio de tensão pode ser escrita como:

$$\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} + f_x = 0$$

$$\frac{\partial \sigma_{yx}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} + f_y = 0$$

$$\frac{\partial \sigma_{ij}}{\partial x_j} + f_i = 0$$

Relação deformação-deslocamento pode ser escrita como:

$$\varepsilon_{xx} = \frac{\partial u_x}{\partial x}; \quad \varepsilon_{yy} = \frac{\partial u_y}{\partial y}; \quad \varepsilon_{xy} = \frac{1}{2} \left(\frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x} \right)$$

$$\varepsilon_{ij} = \frac{1}{2} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right)$$

Relação tensão-deformação pode ser escrita como:

$$\varepsilon_{xx} = \left(\frac{1}{E} \right) \sigma_{xx} + \left(\frac{-v}{E} \right) \sigma_{yy}$$

$$\varepsilon_{yy} = \left(\frac{-v}{E} \right) \sigma_{xx} + \left(\frac{1}{E} \right) \sigma_{yy}$$

$$\varepsilon_{xy} = \frac{1}{2\mu} \sigma_{xy}$$

$$\mu = \frac{E}{2(1+v)}$$

Essas três relações podem ser usadas para chegar na seguinte equação diferencial de deslocamentos:

$$\frac{\partial^2 u_x}{\partial x^2} + \frac{\partial^2 u_x}{\partial y^2} + \frac{1}{1-2v} \left(\frac{\partial^2 u_x}{\partial x^2} + \frac{\partial^2 u_y}{\partial x \partial y} \right) = \frac{-f_x}{\mu}$$

$$\frac{\partial^2 u_y}{\partial x^2} + \frac{\partial^2 u_y}{\partial y^2} + \frac{1}{1-2v} \left(\frac{\partial^2 u_y}{\partial y^2} + \frac{\partial^2 u_x}{\partial x \partial y} \right) = \frac{-f_y}{\mu}$$

$$\frac{\partial^2 u_i}{\partial x_j \partial x_j} + \left(\frac{1}{1-2v} \right) \frac{\partial^2 u_j}{\partial x_i \partial x_j} = \frac{-f_i}{\mu}$$

A solução fundamental dessa equação de deslocamento é dada por:

$$U_{ij}(p, Q) = \frac{1}{8\pi\mu(1-v)} \left\{ (3-4v) \ln \left[\frac{1}{r(p, Q)} \right] \delta_{ij} + \frac{\partial r(p, Q)}{\partial x_i} \frac{\partial r(p, Q)}{\partial x_j} \right\}$$

e para a tração:

$$\begin{aligned} T_{ij}(p, Q) &= \frac{-1}{4\pi(1-\nu)r(p, Q)} \left(\frac{\partial r(p, Q)}{\partial n} \right) \\ &\times \left[(1-2\nu)\delta_{ij} + 2 \frac{\partial r(p, Q)}{\partial x_i} \frac{\partial r(p, Q)}{\partial x_j} \right] \\ &+ \frac{1-2v}{4\pi(1-v)r(p, Q)} \left[\frac{\partial r(p, Q)}{\partial x_j} n_i - \frac{\partial r(p, Q)}{\partial x_i} n_j \right] \end{aligned}$$

Usando um procedimento semelhante ao apresentado para o problema potencial obtemos a equação integral de contorno:

$$\begin{aligned} \begin{bmatrix} u_x(p) \\ u_y(p) \end{bmatrix} &+ \int_{\Gamma} \begin{bmatrix} T_{xx}(p, Q) & T_{xy}(p, Q) \\ T_{yx}(p, Q) & T_{yy}(p, Q) \end{bmatrix} \begin{bmatrix} u_x(Q) \\ u_y(Q) \end{bmatrix} d\Gamma(Q) \\ &= \int_{\Gamma} \begin{bmatrix} U_{xx}(p, Q) & U_{xy}(p, Q) \\ U_{yx}(p, Q) & U_{yy}(p, Q) \end{bmatrix} \begin{bmatrix} t_x(Q) \\ t_y(Q) \end{bmatrix} d\Gamma(Q) \end{aligned}$$

$$u_i(p) + \int_{\Gamma} T_{ij}(p, Q) u_j(Q) \, d\Gamma(Q) = \int_{\Gamma} U_{ij}(p, Q) t_j(Q) \, d\Gamma(Q)$$

Essa equação pode ser derivada e junto com as relações tensão deformação obtém-se a equação que pode ser usada para calcular a tensão:

$$\begin{aligned} & \sigma_{ij}(p) + \int_{\Gamma} \left\{ \frac{(2\mu\nu)}{1-2\nu} \delta_{ij} \frac{\partial T_{mk}(p, Q)}{\partial x_m} \right. \\ & \quad \left. + \mu \left[\frac{\partial T_{ik}(p, Q)}{\partial x_j} + \frac{\partial T_{jk}(p, Q)}{\partial x_i} \right] \right\} u_k(Q) \, d\Gamma(Q) \\ &= \int_{\Gamma} \left\{ \frac{(2\mu\nu)}{1-2\nu} \delta_{ij} \frac{\partial U_{mk}(p, Q)}{\partial x_m} + \mu \left[\frac{\partial U_{ik}(p, Q)}{\partial x_j} + \frac{\partial U_{jk}(p, Q)}{\partial x_i} \right] \right\} t_k(Q) \, d\Gamma(Q) \end{aligned}$$

que pode ser reescrita como:

$$\sigma_{ij}(p) + \int_{\Gamma} S_{kij}(p, Q) u_k(Q) \, d\Gamma(Q) = \int_{\Gamma} D_{kij}(p, Q) t_k(Q) \, d\Gamma(Q)$$

onde os tensores S_{kij} e D_{kij} são dados por:

$$\begin{aligned} S_{kij}(p, Q) &= \frac{\mu}{2\pi(1-\nu)} \left(\frac{1}{r^2} \right) n_i \left[2\nu \frac{\partial r}{\partial x_j} \frac{\partial r}{\partial x_k} + (1-2\nu) \delta_{jk} \right] \\ &+ \frac{\mu}{2\pi(1-\nu)} \left(\frac{1}{r^2} \right) n_j \left[2\nu \frac{\partial r}{\partial x_i} \frac{\partial r}{\partial x_k} + (1-2\nu) \delta_{ik} \right] \\ &+ \frac{\mu}{2\pi(1-\nu)} \left(\frac{1}{r^2} \right) n_k \left[2(1-2\nu) \frac{\partial r}{\partial x_i} \frac{\partial r}{\partial x_j} - (1-4\nu) \delta_{ij} \right] \\ &+ \frac{\mu}{\pi(1-\nu)} \left(\frac{1}{r^2} \right) \left(\frac{\partial r}{\partial n} \right) \left[(1-2\nu) \delta_{ij} \frac{\partial r}{\partial x_k} + \nu \left(\delta_{jk} \frac{\partial r}{\partial x_i} + \delta_{ik} \frac{\partial r}{\partial x_j} \right) \right. \\ & \quad \left. - 4 \frac{\partial r}{\partial x_i} \frac{\partial r}{\partial x_j} \frac{\partial r}{\partial x_k} \right] \\ D_{kij}(p, Q) &= \frac{1}{4\pi(1-\nu)} \left(\frac{1}{r} \right) \left[(1-2\nu) \left(\delta_{jk} \frac{\partial r}{\partial x_i} + \delta_{ik} \frac{\partial r}{\partial x_j} - \delta_{ij} \frac{\partial r}{\partial x_k} \right) \right. \\ & \quad \left. + 2 \frac{\partial r}{\partial x_i} \frac{\partial r}{\partial x_j} \frac{\partial r}{\partial x_k} \right] \end{aligned}$$

15.1 Exercícios

15.1.1 Cilindro pressurizado

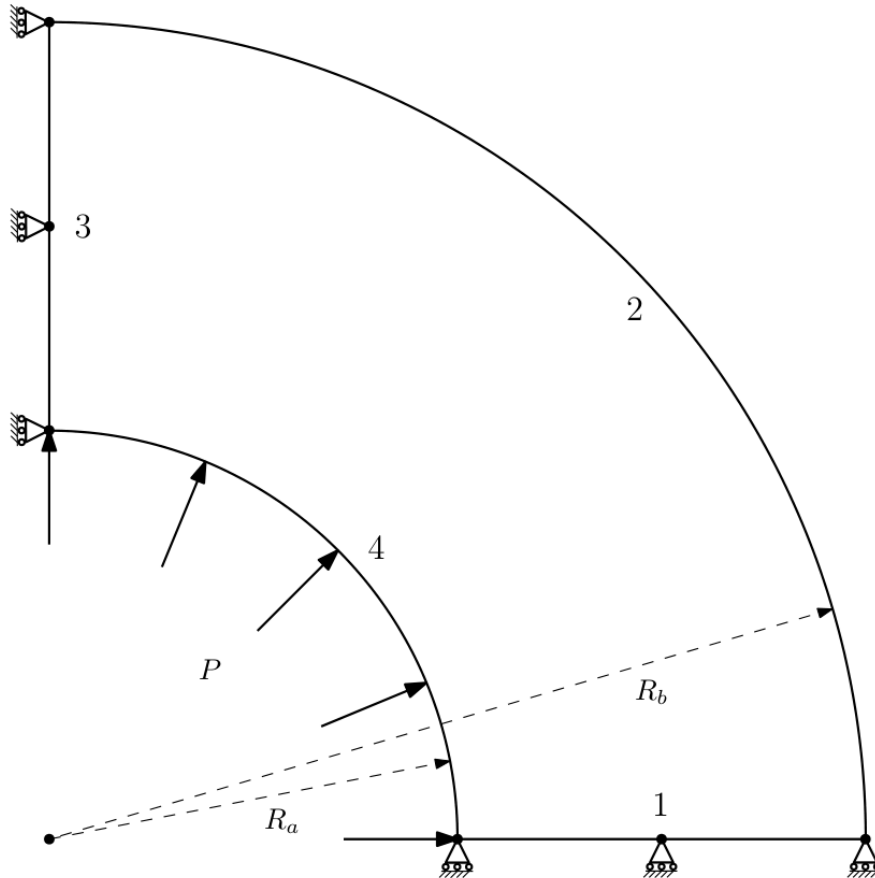
Raio interno $R_a = 50mm$

Raio externo $R_b = 100mm$

Modulo de elasticidade $E = 200GPa$

Poisson $\nu = 0.32$

Pressão $P = 100N/mm$



$$u_r = \frac{(1 + \nu)pr_a^2}{(r_b^2 - r_a^2)E} \left[(1 - 2\nu)r + \frac{r_b^2}{r} \right]$$

$$\sigma_r = \frac{pr_a^2}{r_b^2 - r_a^2} - \frac{r_a^2 r_b^2 p}{r_b^2 - r_a^2} \frac{1}{r^2}$$

$$\sigma_h = \frac{pr_a^2}{r_b^2 - r_a^2} + \frac{r_a^2 r_b^2 p}{r_b^2 - r_a^2} \frac{1}{r^2}$$

15.1.2 Placa com furo

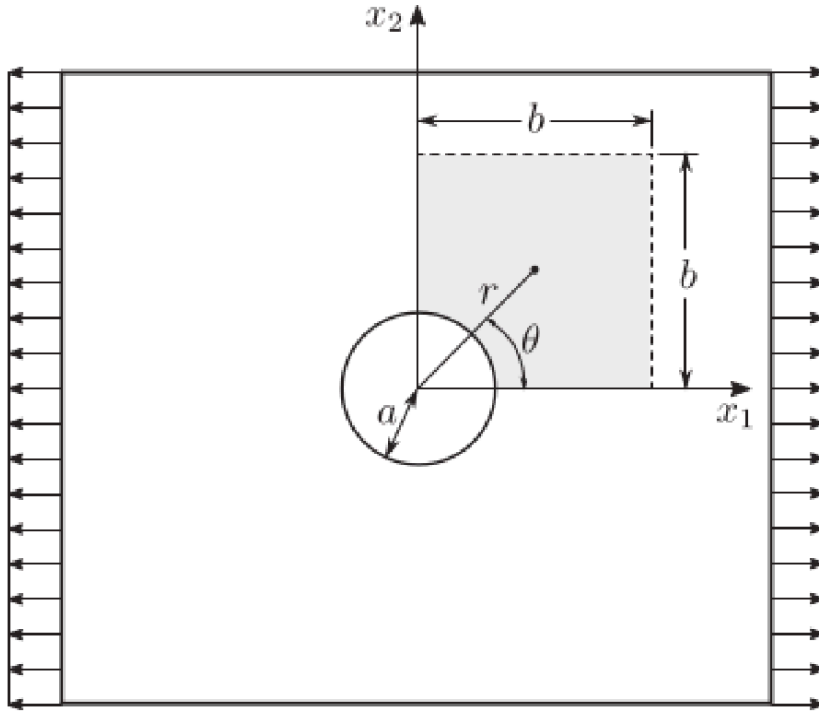
Raio $R = 50mm$

Modulo de elasticidade $E = 100GPa$

Poisson $\nu = 0.25$

Pressão $P = 1N/mm$

Esse problema é de estado plano de tensão. Para ser tratado pelas soluções fundamentais de estado plano de deformação as propriedades tem de ser ajustadas:



$$\nu' = \frac{\nu}{1 + \nu}$$

$$E' = E \left[1 - \frac{\nu'^2}{(1 + \nu')^2} \right]$$

A solução analítica é dada por:

$$\sigma_r = \frac{\sigma}{2} \left(1 - \frac{a^2}{r^2} \right) + \frac{\sigma}{2} \left(1 + \frac{3a^4}{r^2} - \frac{4a^2}{r^2} \right) \cos(2\theta)$$

$$\sigma_\theta = \frac{\sigma}{2} \left(1 + \frac{a^2}{r^2} \right) - \frac{\sigma}{2} \left(1 + \frac{3a^4}{r^4} \right) \cos(2\theta)$$

$$\tau_{r\theta} = -\frac{\sigma}{2} \left(1 - \frac{3a^4}{r^4} + \frac{4a^2}{r^2} \right) \sin(2\theta)$$

15.1.3 Viga

Uma viga com um carregamento parabólico $t_2(y) = -\frac{P}{2I} \left(\frac{D^2}{4} - y^2 \right)$ tem solução analítica:

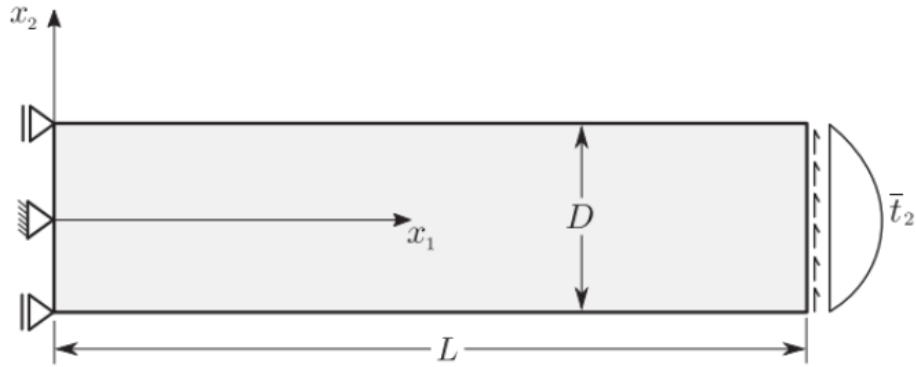
Comprimento $L = 48mm$

Altura $D = 12mm$

Modulo de elasticidade $E = 300GPa$

Poisson $\nu = 0.3$

Pressão $P = 1000N$



$$u_1(x, y) = -\frac{Py}{6EI} \left[(6L - 3x)x + (2 + v) \left(y^2 - \frac{D^2}{4} \right) \right]$$

$$u_2(x, y) = \frac{P}{6EI} \left[(3v)y^2(L - x) + (4 + 5v) \frac{D^2x}{4} + (3L - x)x^2 \right]$$

$$\sigma_{xx}(x, y) = -\frac{P(L - x)y}{I}$$

$$\tau_{xy}(x, y) = -\frac{P}{2I} \left(\frac{D^2}{4} - y^2 \right)$$

gravação

16 Propgeo 3D

<https://1drv.ms/f/s!AmfyGvdmTYongqYn5kjjlZaMHR9h2w?e=z0sXvU>

gravação